

UNIVERSITÉ CLERMONT AUVERGNE
Clermont-Ferrand

École Universitaire de Physique et d'Ingénierie

RAPPORT DE STAGE

1^{ère} année de Master

Spécialité : Mécatronique

Rigon Ajvazi

Détection et estimation 3D d'une balle rapide par caméra événementielle

Période de stage	du 1 ^{er} avril 2026 au 25 juillet 2026
Organisme d'accueil	Institut Pascal, Université Clermont Auvergne / CNRS
Soutien financier	Bourse CIR ITPS, International Research Centre on Innovative Transportation and Production Systems, I-SITE CAP 20-25
Tuteurs de stage	Sébastien Lengagne, Céline Teulière, Antonio Claudio de Sousa dos Santos Filho
Tuteur académique	Benoit Thuilot
Date de soutenance	4 septembre 2026

Soutien financier

Remerciements

Je tiens tout d'abord à remercier mon tuteur de stage, Sébastien Lengagne, ainsi que Céline Teulière et Antonio Claudio de Sousa dos Santos Filho, pour leur encadrement, leur disponibilité, leur aide et les échanges qui ont orienté mon travail tout au long du stage. Leur accompagnement m'a permis de mieux comprendre les enjeux de la perception robotique rapide et de situer mon travail dans un contexte de recherche appliquée.

Je remercie également mon tuteur académique, Benoit Thuilot, pour le suivi du stage et pour les conseils apportés dans la structuration du travail. Je remercie plus largement les membres de l'Institut Pascal et les personnes avec lesquelles j'ai pu échanger au cours du stage, pour leur accueil.

Enfin, je remercie l'Université Clermont Auvergne et l'Institut Pascal de m'avoir permis de réaliser ce stage dans un cadre associant robotique, vision événementielle et apprentissage par renforcement.

Ce travail a bénéficié du soutien du Centre International de Recherche *Innovative Transportation and Production Systems* (CIR ITPS) de l'I-SITE CAP 20-25, par l'attribution d'une bourse CIR ITPS.

Résumé du sujet de stage

Ce rapport présente le travail réalisé pendant un stage effectué du 1^{er} avril 2026 au 25 juillet 2026 à l'Institut Pascal, unité mixte de recherche de l'Université Clermont Auvergne et du CNRS. Le sujet porte sur la détection et l'estimation 3D d'une balle rapide à partir d'une caméra événementielle DVXplorer, dans la perspective d'une tâche robotique d'interception.

L'objectif du stage est de construire une chaîne de perception capable de transformer le flux d'événements produit par la caméra en une position de balle exploitable par un robot. Cette information doit être suffisamment rapide et cohérente pour préparer une interception, tout en tenant compte des contraintes de calibration, de bruit et de mouvement rapide. Le rapport présente le contexte de l'organisme d'accueil, l'environnement de travail, les choix algorithmiques, les étapes de calibration intrinsèque et extrinsèque, la validation en simulation, les premiers essais d'intégration sur le robot UR3e réel et les perspectives pour l'interception complète.

Mots clés : caméra événementielle, robotique, estimation 3D, détection de balle, ROS 2, Isaac Lab, apprentissage par renforcement.

Table des matières

Remerciements	i
Résumé du sujet de stage	ii
1 Introduction	1
2 Présentation de l'organisme d'accueil	2
2.1 Université Clermont Auvergne	2
2.2 Institut Pascal	3
2.3 Activités et axes de recherche	3
2.4 Centre International de Recherche CIR ITPS	4
2.5 Organisation interne	4
2.6 Unité d'accueil : l'axe ISPR et les équipes MACCS et ComSee	5
3 Perception événementielle de la balle	8
3.1 Poste occupé et étude confiée	8
3.2 Environnement technique	8
3.3 Chaîne de traitement	9
3.4 Calibration intrinsèque et repère caméra	10
3.5 Méthode par circle fitting	13
3.6 Méthode par trace événementielle	22
3.7 Simulation et validation	25
4 Intégration robotique et contrôle	30
4.1 Calibration extrinsèque par mire événementielle	30
4.2 Apprentissage d'une politique d'interception en simulation	37
4.3 Boucle d'interception temps réel	39

4.4 Premiers essais sur le robot réel	40
4.5 Défis pour l'intégration perception-robot	40
5 Conclusion	42
A Expressions des régressions de trajectoire par moindres carrés	43
B Pipeline détaillé de la détection par circle fitting	44
C Détails de la reconstruction du ruban de trace	45
D Paramètres des séquences simulées	49
E Résultats numériques de la calibration caméra-robot	50
F Récapitulatif des réalisations du stage	52
Bibliographie et webographie	54

Chapitre 1

Introduction

Le sujet du stage s'appuie sur une situation simple à décrire : une personne lance une balle vers un robot manipulateur, et le robot doit l'attraper avec un filet à papillon fixé à son effecteur. Pour réussir cette interception, le robot doit connaître très vite la position de la balle, puis utiliser cette information pour placer le filet au bon endroit au bon moment.

Le stage a été réalisé à l'Institut Pascal, au sein de l'Université Clermont Auvergne, du 1^{er} avril 2026 au 25 juillet 2026. Il concerne principalement la partie perception de cette chaîne. La balle est observée avec une caméra événementielle DVXplorer, c'est-à-dire un capteur qui ne fournit pas des images complètes à fréquence fixe, mais des événements datés lorsqu'un changement local de luminosité est détecté. Ce fonctionnement est adapté à une balle rapide, car il donne une information temporelle fine et limite la quantité de données à traiter.

Le problème central est donc de transformer un flux d'événements en une position 3D de balle fiable, avec une latence assez faible pour rester utile à la commande du robot. Une erreur de profondeur ou un retard de quelques millisecondes peut déplacer fortement le point d'interception. La perception doit donc être rapide, robuste au bruit, cohérente dans le temps et compatible avec la calibration entre la caméra et le robot.

Les objectifs généraux du projet sont les suivants :

- obtenir une position 3D exploitable de la balle à partir d'une caméra événementielle ;
- conserver une latence faible pour rendre l'information utilisable pendant le mouvement de la balle ;
- assurer la cohérence entre la perception, la calibration caméra-robot et la commande d'interception ;
- identifier les limites de la chaîne avant son intégration complète sur le robot réel.

Ce rapport commence par présenter l'organisme d'accueil, l'Université Clermont Auvergne et l'Institut Pascal, ainsi que l'axe de recherche et les équipes qui ont accueilli et encadré le stage. La suite est organisée autour de deux grands blocs techniques. Le premier concerne la perception événementielle de la balle : l'environnement de travail, la chaîne de traitement, la calibration de la caméra, les deux méthodes d'estimation 3D développées et leur validation en simulation. Le second concerne l'intégration robotique : la calibration entre la caméra et le robot, l'apprentissage d'une politique d'interception en simulation et les premiers essais réalisés sur le robot réel. Le rapport se termine par un bilan technique et personnel du stage.

Chapitre 2

Présentation de l'organisme d'accueil

2.1 Université Clermont Auvergne

L'Université Clermont Auvergne (UCA) est un établissement public expérimental d'enseignement supérieur et de recherche. Elle est issue de deux étapes récentes de structuration du site clermontois : la fusion, en 2017, de l'université Blaise-Pascal et de l'université d'Auvergne, les deux universités historiques de Clermont-Ferrand, puis l'adoption du statut d'établissement public expérimental. La nouvelle UCA, sous ce statut, est née le 1^{er} janvier 2021 dans le prolongement de cette structuration et de la démarche I-SITE CAP 20-25 ; elle intègre notamment Clermont Auvergne INP, qui regroupe les écoles d'ingénieurs du site. Elle rassemble des activités de formation, de recherche et d'innovation sur plusieurs sites à Clermont-Ferrand et dans la région Auvergne.

L'UCA indique rassembler environ 36 000 étudiants, 350 formations et 47 structures de recherche [4]. Elle s'organise autour de six instituts qui regroupent les composantes de formation et de recherche par grands domaines. Cette organisation permet de faire le lien entre enseignement supérieur, laboratoires, plateformes expérimentales et partenaires socio-économiques.

TABLE 2.1 – Carte d'identité synthétique de l'Université Clermont Auvergne et de l'organisme d'accueil.

Élément	Information
Nom de l'établissement	Université Clermont Auvergne (UCA)
Statut	Établissement public expérimental à caractère scientifique, culturel et professionnel
Adresse du siège	49 boulevard François-Mitterrand, CS 60032, 63001 Clermont-Ferrand Cedex 1
Téléphone	04 73 17 79 79
SIRET du siège	130 028 061 00013
Organisme d'accueil du stage	Institut Pascal, UMR 6602 UCA/CNRS
Adresse de l'Institut Pascal	Campus universitaire des Cézeaux, 4 avenue Blaise Pascal, TSA/CS 60026, 63178 Aubière Cedex
Effectif de l'Institut Pascal	Environ 400 personnes
Activité principale	Formation, recherche et innovation, en particulier en sciences de l'ingénierie pour l'Institut Pascal
Capital social	Non applicable pour un établissement public

2.2 Institut Pascal

L’Institut Pascal est l’unité de recherche dans laquelle le stage s’est déroulé. Il s’agit d’une unité mixte de recherche et de formation interdisciplinaire, UMR 6602, placée sous la tutelle de l’Université Clermont Auvergne et du CNRS, avec le CHU de Clermont-Ferrand comme tutelle secondaire. L’institut est également membre de Clermont Auvergne INP. Il couvre les sciences de l’ingénierie du site clermontois : génie des procédés, mécanique, robotique, physique des sciences de l’information et ingénierie de la santé.

L’Institut Pascal a été créé le 1^{er} janvier 2012 par le regroupement de laboratoires du site clermontois, dont le LASMEA pour l’électronique, l’automatique et la vision, le LaMI pour la mécanique et les ingénieries et le LGCB pour le génie chimique et biochimique, avec l’objectif de rassembler des compétences complémentaires en sciences de l’ingénierie et de favoriser des projets interdisciplinaires [5]. Ce périmètre s’est ensuite élargi par fusions successives, en 2017 puis en 2021, jusqu’à couvrir l’essentiel des sciences de l’ingénierie et des systèmes du site. L’institut rassemble environ 400 personnes, dont près de 150 enseignants-chercheurs et chercheurs et presque autant de doctorants [5], et se positionne sur des recherches allant du développement de connaissances fondamentales jusqu’à leur intégration dans des systèmes. Ses domaines d’application principaux sont la mobilité et les transports, l’usine et l’industrie 4.0, ainsi que l’hôpital du futur. Cette orientation explique la place importante donnée aux systèmes robotiques, à la perception, à la commande et aux plateformes expérimentales.

Le stage a eu lieu sur le campus universitaire des Cézeaux, à Aubière. Ce campus regroupe plusieurs composantes scientifiques et techniques de l’UCA, ainsi que des laboratoires et plateformes utiles aux travaux en robotique, ce qui facilite les échanges entre étudiants, enseignants-chercheurs, doctorants, ingénieurs et personnels techniques.

2.3 Activités et axes de recherche

Les activités de l’Institut Pascal concernent principalement la recherche académique, les projets collaboratifs, les contrats avec des partenaires industriels, la participation à des réseaux scientifiques et l’appui à la formation par la recherche.

Ces activités s’appuient sur cinq axes de recherche :

- Génie des Procédés, Énergétique et Biosystèmes (GePEB) ;
- Image, Systèmes de Perception, Robotique (ISPR) ;
- Mécanique, Génie Mécanique, Génie Civil, Génie Industriel (M3G) ;
- Photonique, Ondes, Nanomatériaux (PHOTON) ;
- Thérapies Guidées par l’Image (TGI).

Ces activités se déclinent dans trois grands domaines d’application. Le premier concerne la mobilité et les transports des biens et des personnes, avec des systèmes plus sûrs, plus autonomes ou plus efficaces s’appuyant sur la mécanique, la robotique, la perception et la modélisation. Le deuxième concerne l’usine et l’industrie 4.0, avec des problématiques de production, d’automatisation, de robotique collaborative, de contrôle et d’interaction humain-machine. Le troisième concerne l’hôpital du futur, dans lequel l’image, les systèmes guidés et les dispositifs d’assistance jouent un rôle important.

L’Institut Pascal participe également à des dispositifs de partenariat structurants, associés à des projets soutenus par l’ANR, l’Union Européenne, la Région Auvergne-Rhône-Alpes ou le programme CAP 20-25, ainsi qu’à des collaborations industrielles comme FactoLab, laboratoire commun lié à Michelin. Cette proximité avec des problématiques réelles explique que les travaux

de recherche ne se limitent pas à des modèles théoriques : ils cherchent souvent à produire des solutions testables sur des plateformes expérimentales.

2.4 Centre International de Recherche CIR ITPS

Le Centre International de Recherche *Innovative Transportation and Production Systems* (CIR ITPS) est l'un des dispositifs de l'I-SITE CAP 20-25. Il a pour rôle de fédérer les activités de recherche du site clermontois autour des systèmes innovants pour les transports et la production, de renforcer la visibilité internationale du site et de contribuer au déploiement territorial des activités scientifiques et de valorisation [2].

Ses thématiques couvrent plusieurs enjeux liés à la mobilité, à la production et aux systèmes autonomes : autonomie des robots mobiles et des véhicules, développement de cobots mobiles, planification des systèmes de production ou de transport, nouveaux vecteurs énergétiques et acceptabilité des nouvelles technologies [2]. Le CIR ITPS rassemble ainsi des compétences qui vont de la mécanique et de la robotique jusqu'à l'aide à la décision, aux matériaux durables et aux sciences humaines.

Le sujet de ce stage s'inscrit principalement dans la partie robotique et perception de ce cadre. L'objectif final, qui consiste à permettre à un robot d'intercepter une balle rapide, correspond à une problématique de système dynamique où la perception, la commande et l'expérimentation doivent être étudiées ensemble. La caméra événementielle répond au besoin d'une perception rapide, tandis que la partie robotique prépare l'utilisation de cette information pour une action physique.

Le stage présenté dans ce rapport a bénéficié d'une bourse CIR ITPS. Cette aide rattache le travail à un cadre de recherche appliquée où les méthodes développées doivent pouvoir être testées sur des systèmes réels ou proches du réel. La mention de ce soutien et l'utilisation des logos CIR ITPS et I-SITE CAP 20-25 répondent aux consignes officielles applicables aux rapports de stage soutenus par ce centre [3].

2.5 Organisation interne

La structure interne de l'Institut Pascal repose sur des axes scientifiques qui correspondent à de grands ensembles de compétences. Cette organisation permet de regrouper les chercheurs et personnels techniques par disciplines, tout en favorisant des projets transversaux. Dans le cas de mon stage, la perception événementielle mobilise plusieurs notions complémentaires : géométrie 3D, traitement du signal, programmation temps réel, robotique et apprentissage. Le sujet se rattache à l'axe ISPR, présenté en détail dans la section suivante avec les deux équipes qui ont encadré le travail. La figure 2.1 donne une lecture simplifiée de cette organisation et de la place du stage.

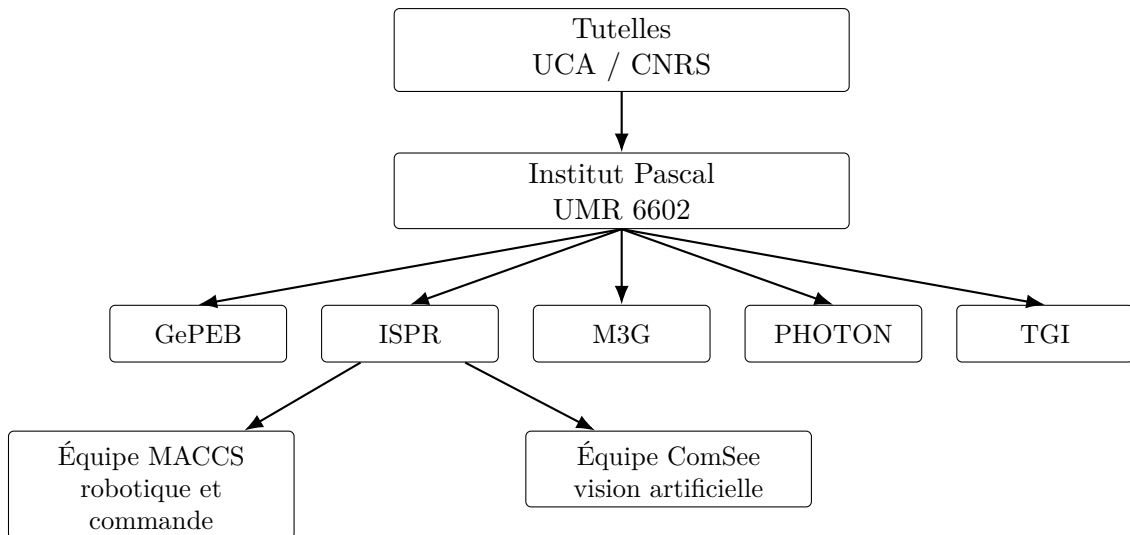


FIGURE 2.1 – Organigramme simplifié situant le stage dans l’organisation scientifique de l’Institut Pascal.

2.6 Unité d’accueil : l’axe ISPR et les équipes MACCS et ComSee

2.6.1 L’axe ISPR

Le stage s’est déroulé au sein de l’axe Image, Systèmes de Perception, Robotique (ISPR), l’un des cinq axes de recherche présentés plus haut. Cet axe travaille dans le domaine de la perception et de la vision artificielle pour la commande de systèmes robotiques. Son objectif scientifique est de développer des concepts théoriques, méthodologiques et architecturaux permettant à un système de percevoir son environnement, d’en extraire une information fiable, puis d’utiliser cette information pour agir [6]. Le sujet du stage, transformer le flux d’une caméra événementielle en une position de balle exploitable pour une interception robotique, est une déclinaison directe de cette question.

L’axe regroupe des enseignants-chercheurs, des chercheurs, des ingénieurs et des doctorants, et il est organisé en quatre thèmes complémentaires [6] :

- ComSee (*Computers that See*), consacré à la vision par ordinateur ;
- DREAM, consacré aux architectures embarquées et aux systèmes multi-capteurs ;
- MACCS (Modélisation, Autonomie et Contrôle dans les Systèmes Complexes), consacré à la modélisation et à la commande des systèmes robotiques ;
- PerSyst, consacré aux systèmes de perception.

Ces thèmes partagent des applications communes : véhicules et robots mobiles autonomes, robotique de manipulation, perception embarquée et interaction avec l’environnement. Ils s’appuient sur les plateformes expérimentales du site clermontois et s’inscrivent dans les programmes structurants de l’institut et du CIR ITPS évoqués précédemment, pour lesquels la robotique autonome est une thématique fédératrice.

2.6.2 Les équipes MACCS et ComSee

Au sein de l'axe ISPR, le stage a été co-encadré par des chercheurs de deux équipes dont les compétences correspondent aux deux volets du sujet.

L'équipe MACCS s'intéresse principalement à la modélisation et à la commande de robots mobiles et de manipulateurs, à la vision robotique, à l'asservissement visuel et aux comportements anticipatifs [7]. C'est la composante robotique du stage : le choix du robot UR3e, la définition de la tâche d'interception, l'apprentissage de la politique de commande et les contraintes de sécurité des essais réels relèvent de cette culture scientifique. Mon tuteur principal, Sébastien Lengagne, appartient à cette équipe, de même qu'Antonio Claudio de Sousa dos Santos Filho, doctorant de l'équipe qui a participé à mon encadrement et dont les travaux servent de point de départ à mon stage. Mon tuteur académique, Benoit Thuilot, en est également membre ; ses travaux portent sur la commande et la robotique mobile [7].

L'équipe ComSee est consacrée à la vision par ordinateur [8]. Ses travaux portent notamment sur la calibration de capteurs et la reconstruction 3D sans contact, la reconnaissance et le suivi temps réel de motifs visuels, y compris par apprentissage, et la localisation de véhicules mobiles. La partie perception du stage, détection de la balle dans le flux événementiel, estimation 3D et calibration de la caméra, s'inscrit directement dans cette thématique, encadrée par Céline Teulière.

Ces travaux de thèse [1] portaient sur la vision, l'apprentissage par renforcement et le transfert de compétences pour une tâche robotique dynamique. Ils m'ont assuré une continuité concrète : un objectif applicatif déjà défini, un robot disponible et de premières analyses sur lesquelles m'appuyer.

2.6.3 Environnement humain et technique du stage

Au quotidien, le stage s'est déroulé sur le campus des Cézeaux, dans l'environnement d'expérimentation robotique de l'équipe, où sont installés le robot UR3e et la caméra événementielle DVXplorer utilisés pour les essais réels. L'environnement matériel mis à disposition comprend le bras manipulateur et son contrôleur, la caméra événementielle, un poste de travail pour le développement et le traitement temps réel, une station de calcul équipée d'un processeur graphique pour la simulation et l'apprentissage par renforcement, ainsi qu'un accès à l'impression 3D pour la fabrication des pièces mécaniques du projet, comme le support de mire et le support du filet décrits dans la suite du rapport.

L'environnement humain est celui d'une équipe de recherche : enseignants-chercheurs, doctorants et stagiaires partagent les mêmes espaces, ce qui facilite les échanges informels et l'entraide technique. Le suivi du stage s'est fait par des points réguliers avec les encadrants, au cours desquels les résultats intermédiaires étaient discutés et les orientations suivantes décidées. Cette double appartenance, commande côté MACCS et vision côté ComSee, s'est révélée particulièrement adaptée au sujet, qui exige en permanence de raisonner sur les deux bouts de la chaîne à la fois.

Enfin, l'unité n'est pas isolée dans l'institut : les questions mécaniques du projet, comme la conception des supports imprimés en 3D ou la rigidité du montage de calibration, touchent aux compétences de l'axe M3G, et l'institut anime des programmes transversaux qui relient les axes entre eux autour des machines et robots intelligents [5].

Positionnement du sujet dans l'organisme

Le sujet de stage se situe au croisement de plusieurs objectifs de l'Institut Pascal : développer des méthodes de perception, les tester sur des systèmes rapides, et préparer leur utilisation dans un contexte robotique. Il ne s'agit pas uniquement de détecter une balle dans une image. Il faut produire une estimation 3D robuste, temporellement cohérente, suffisamment rapide et compatible avec une commande robotique. Le sujet est ainsi représentatif d'une problématique de mécatronique : le résultat dépend à la fois du capteur, de l'algorithme, du modèle géométrique, du logiciel temps réel et du système robotique.

Chapitre 3

Perception événementielle de la balle

3.1 Poste occupé et étude confiée

Pendant le stage, j'ai occupé une fonction de développement et d'expérimentation autour d'une chaîne de perception robotique. Le travail confié consistait à étudier comment détecter une balle rapide dans un flux événementiel, estimer sa position 3D, analyser les limites des méthodes mises en place et préparer l'utilisation de cette information par une partie robotique.

La tâche finale à réaliser est de permettre à un robot à 6 degrés de liberté, ici un UR3e, d'attraper une balle lancée vers lui. Pour détecter la balle avec une latence suffisamment faible, le système s'appuie sur une caméra événementielle, dont les données doivent être transformées en une information de position exploitable par le robot.

Les tâches principales ont été les suivantes :

- comprendre le fonctionnement de la caméra événementielle et le format des données produites ;
- développer une chaîne C++/ROS 2 [11] de filtrage, regroupement et estimation de la balle ;
- comparer une méthode par ajustement de cercle avec une méthode par analyse de trace ;
- utiliser des séquences simulées pour disposer d'une vérité terrain et analyser les erreurs ;
- identifier les besoins de calibration entre la caméra et le robot ;
- préparer le lien avec une politique d'interception apprise en simulation.

L'annexe F donne un récapitulatif synthétique, par thème, de l'ensemble des réalisations techniques produites pendant le stage.

3.2 Environnement technique

L'environnement technique du stage combine plusieurs outils matériels et logiciels. La caméra DVXplorer [9] fournit les événements utilisés par les méthodes de détection. Les données peuvent provenir de la caméra réelle ou de séquences enregistrées, ce qui permet de répéter les essais et de comparer plusieurs versions de l'algorithme dans les mêmes conditions. Le traitement est réalisé principalement en C++, car la tâche impose une vitesse de calcul élevée et une latence faible : l'information produite doit rester utilisable pour une interception rapide. ROS 2 [11] est utilisé pour publier les résultats et préparer l'intégration avec la chaîne robotique. Une interface adaptée au projet est également développée avec raylib [13], une bibliothèque C/C++ légère destinée aux affichages graphiques interactifs, afin de visualiser les événements, les détections et

les estimations 3D pendant les essais.

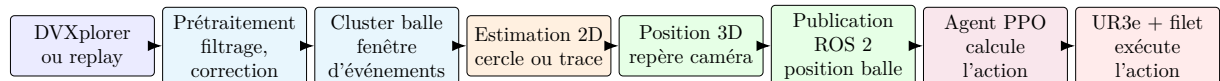
TABLE 3.1 – Principaux moyens techniques mobilisés pendant le stage.

Élément	Utilisation dans le stage
Caméra événementielle DVXplorer [9]	Acquisition d'événements à faible latence pour suivre une balle rapide.
Séquences enregistrées	Relecture des essais pour tester les algorithmes de manière reproductible.
C++ et ROS 2 [11]	Traitement rapide à faible latence, publication des résultats et préparation de l'intégration robotique.
OpenCV [12]	Correction géométrique, calibration intrinsèque et calculs associés à la projection caméra.
raylib [13]	Bibliothèque graphique légère utilisée pour créer une interface adaptée au projet et visualiser les résultats.
Isaac Lab [15] et PPO [18]	Étude de la partie robotique et de l'apprentissage d'une politique d'interception.
Visualisations 2D/3D	Diagnostic des clusters, des trajectoires et des erreurs de profondeur.

Cette combinaison d'outils impose de garder une cohérence entre plusieurs repères et plusieurs échelles de temps. Les événements ont leurs propres timestamps, les estimations 3D sont exprimées dans le repère caméra, le robot travaille dans son repère de base, et la simulation utilise encore ses propres conventions. Une partie importante du travail consiste donc à vérifier que les informations produites par la perception pourront être utilisées sans ambiguïté dans la suite de la chaîne.

3.3 Chaîne de traitement

La chaîne de traitement transforme le flux brut d'événements en une position 3D exploitable par la partie robotique. Les événements proviennent soit de la caméra DVXplorer [9], soit de séquences enregistrées pour répéter les essais. Ils sont d'abord filtrés, corrigés avec la calibration de la caméra, conservés dans une fenêtre temporelle courte puis échantillonnés afin de limiter la charge de calcul. Les points restants sont ensuite regroupés spatialement pour isoler la balle, puis analysés par une méthode d'estimation 2D, par cercle ou par trace. Cette estimation est convertie en position 3D, visualisée dans l'interface et publiée dans ROS 2 [11]. La position est ensuite envoyée à l'agent PPO, qui calcule les actions à appliquer au robot pour placer le filet d'interception.



Principe : ne pas reconstruire une image complète, mais garder les événements utiles pour produire rapidement une position de balle exploitable par la commande robotique.

FIGURE 3.1 – Vue simplifiée de la chaîne de traitement C++/ROS 2 [11], depuis l'estimation 3D de la balle jusqu'à l'action calculée par l'agent PPO.

3.4 Calibration intrinsèque et repère caméra

Les méthodes de perception présentées dans la suite de ce chapitre convertissent une détection dans l'image en une position 3D exprimée dans le repère de la caméra. Cette conversion repose entièrement sur le modèle optique du capteur : les focales f_x et f_y , le centre optique (c_x, c_y) et les coefficients de distorsion. Une erreur sur ces paramètres se propage directement aux positions 3D estimées, en particulier à la profondeur, qui dépend linéairement de la focale. Avant tout essai quantitatif, il faut donc estimer ces paramètres : c'est le rôle de la calibration intrinsèque. La calibration entre la caméra et le robot, qui répond à un besoin différent, est traitée plus loin, dans la partie consacrée à l'intégration robotique.

La difficulté vient de la nature du capteur. Les méthodes classiques détectent les coins d'un échiquier dans des images complètes, alors qu'une caméra événementielle ne produit que des changements locaux de luminosité. Une première calibration a été réalisée avec les outils officiels d'iniVation [10], qui reconstruisent une image à partir des événements accumulés : il faut alors bouger constamment la mire d'échiquier devant la caméra afin de générer assez d'événements pour obtenir une image exploitable, comme le montre la figure 3.2. Cette première calibration donne les paramètres suivants, avec une erreur de reprojection d'environ 0.486 px :

$$f_x \approx 761 \text{ px}, \quad f_y \approx 754 \text{ px}, \quad c_x \approx 305 \text{ px}, \quad c_y \approx 196 \text{ px}.$$

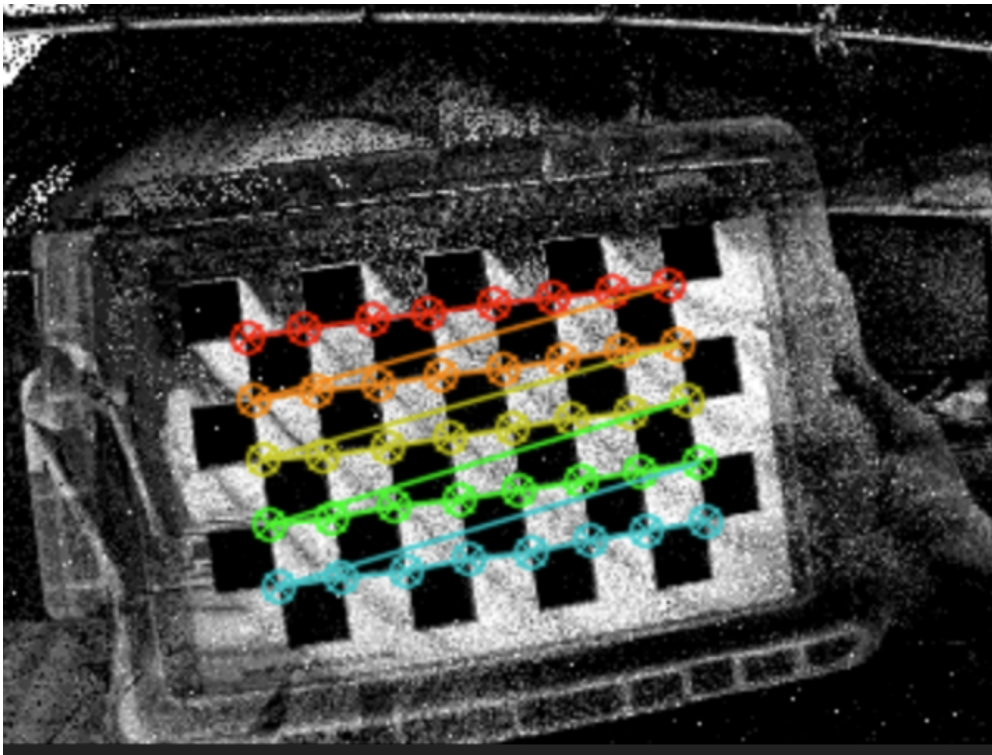


FIGURE 3.2 – Première calibration intrinsèque par échiquier : les coins de la mire sont recherchés dans une image reconstruite à partir des événements accumulés pendant le déplacement de la mire.

Cette procédure fonctionne, mais elle présente deux limites : l'image reconstruite reste bruitée, et l'acquisition est contraignante puisque la mire doit être déplacée en continu pendant toute la calibration. Ces limites ont motivé le développement d'un nouvel outil de calibration dont le

principe est pensé directement pour une caméra événementielle. L'idée est d'inverser le problème : au lieu de créer des variations de luminosité en déplaçant une mire imprimée, c'est la mire elle-même qui produit ces variations. Des disques lumineux clignotant sur fond noir sont affichés sur un écran ; chaque changement d'état génère des événements nets exactement aux positions utiles, sans aucun mouvement. Cette approche est plus simple à mettre en œuvre, et elle a réduit de manière importante le temps nécessaire pour réaliser une calibration exploitable.

La mire retenue, illustrée par la figure 3.3, est une grille de 4×5 disques rendue volontairement asymétrique par un disque manquant et un disque plus grand servant d'ancre, soit 19 points détectables. Cette asymétrie lève les ambiguïtés d'orientation : quelle que soit la pose de l'écran, chaque disque détecté peut être associé sans confusion à un point physique connu de la mire. Un léger gradient radial de luminosité peut aussi être appliqué aux disques, afin d'adoucir l'activité générée à leurs bords tout en gardant un centre bien marqué.

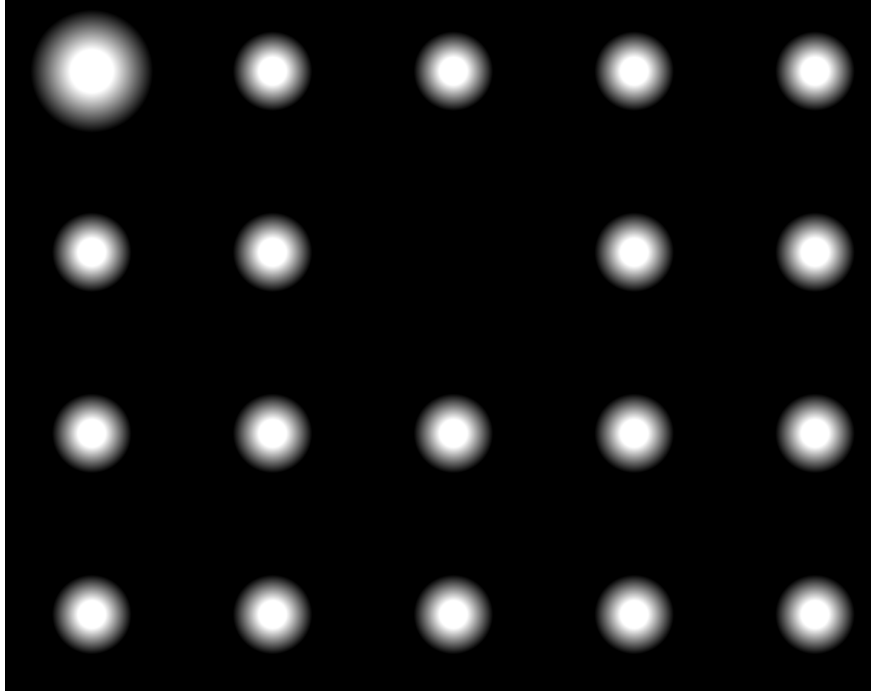


FIGURE 3.3 – Mire de calibration affichée sur un écran : grille asymétrique de disques lumineux, avec un disque manquant et un disque plus grand servant d'ancre. Les disques clignotent pour générer une activité événementielle exploitable par la caméra DVXplorer [9].

La détection des disques repose sur une image d'activité. Pendant une courte fenêtre temporelle $\mathcal{W}$ entourant un changement d'état de la mire, les événements positifs et négatifs sont accumulés pixel par pixel :

$$A(u, v) = \sum_{k \in \mathcal{W}} \mathbf{1}[(u_k, v_k) = (u, v)].$$

Les zones où l'activité est forte correspondent aux disques qui viennent de changer de luminosité. Elles sont seuillées puis regroupées en composantes connexes, et le centre de chaque disque est estimé par barycentre pondéré de l'activité :

$$u_c = \frac{\sum u A(u, v)}{\sum A(u, v)}, \quad v_c = \frac{\sum v A(u, v)}{\sum A(u, v)}.$$

Ce centre est plus stable qu'un simple centre de boîte englobante, car il exploite la distribution complète de l'activité du blob. Comme la taille physique de l'écran est connue, chaque disque possède aussi une coordonnée 3D connue dans le plan de la mire, exprimée en millimètres. Chaque acquisition fournit alors les correspondances entre les points de la mire $(X, Y, 0)$ et leurs centres image (u, v) . Cette mire et cette détection sont réutilisées au chapitre suivant pour la calibration entre la caméra et le robot.

Les observations accumulées sur plusieurs poses de la mire alimentent ensuite la calibration OpenCV [12], qui estime la matrice intrinsèque, les coefficients de distorsion et l'erreur de reprojection. Une sélection robuste peut être activée : plusieurs sous-ensembles d'observations sont testés, les vues cohérentes sont conservées, puis la calibration finale est relancée sur les vues retenues. Cette étape permet d'écarter automatiquement des captures où l'association de points est moins fiable.

Dans l'essai conservé, la calibration robuste utilise 17 observations de mire sur une image 640×480 , avec une erreur RMS d'environ 0.15 px. Elle produit notamment :

$$f_x \approx 526.4 \text{ px}, \quad f_y \approx 525.8 \text{ px}, \quad c_x \approx 331.3 \text{ px}, \quad c_y \approx 242.7 \text{ px}.$$

Ces valeurs ne doivent pas être comparées directement à celles de la calibration par échiquier présentée plus haut. Entre les deux essais, la monture de la caméra a été touchée, ce qui a pu modifier le réglage mécanique et donc les paramètres observés. L'écart entre les deux calibrations ne permet donc pas, à lui seul, de conclure qu'une méthode est meilleure que l'autre ; il montre surtout que les deux acquisitions ne correspondent pas exactement au même état matériel.

Les coefficients de distorsion radiale estimés sont environ $k_1 = -0.402$ et $k_2 = 0.170$, avec une distorsion tangentielle fixée à zéro et k_3 fixé à zéro. Le résultat est ensuite directement utilisé par la chaîne de perception, notamment pour la correction de distorsion et le calcul de la profondeur.

Un test complémentaire a été ajouté pour vérifier que cette calibration ne fonctionne pas seulement sur la mire utilisée pour l'estimation. Une cible carrée à quatre points est affichée dans plusieurs positions et tailles, puis les distances reconstruites sont comparées aux dimensions réelles de l'écran. Sur l'exemple retenu, les quatre carrés sont validés, avec une erreur RMS d'environ 0.13 px et une erreur moyenne d'espacement d'environ 0.16 mm. La figure 3.4 montre le montage utilisé et une sortie de validation.

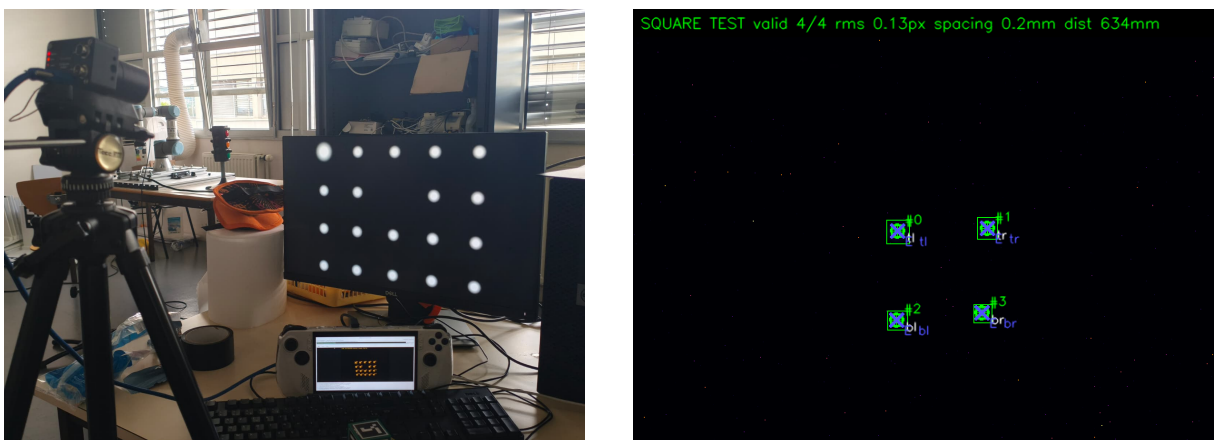


FIGURE 3.4 – Calibration intrinsèque par mire événementielle. À gauche, la caméra observe une mire clignotante affichée sur écran ; à droite, le test carré vérifie la cohérence de la calibration obtenue.

Cette procédure étant rapide à rejouer, la caméra a été recalibrée avant les essais réels sur le robot : la calibration utilisée pour les essais du chapitre suivant atteint une erreur RMS d'environ 0,149 px avec 17 observations de mire.

Pour les essais en simulation, la caméra virtuelle est définie avec une calibration synthétique différente :

$$f_x = f_y = 520 \text{ px}, \quad c_x = 320 \text{ px}, \quad c_y = 240 \text{ px}.$$

3.5 Méthode par circle fitting

3.5.1 Signal événementiel et polarités

Une caméra événementielle ne transmet pas des images complètes à fréquence fixe. Chaque pixel fonctionne de manière indépendante et déclenche un événement lorsqu'il détecte une variation suffisante de luminosité. Un événement contient donc quatre informations principales :

$$e = (x, y, t, p),$$

où x et y sont les coordonnées du pixel, t est le timestamp, et p est la polarité. La polarité indique le signe de la variation de luminosité : une polarité positive correspond à une augmentation locale de luminosité, tandis qu'une polarité négative correspond à une diminution. Pour une balle en mouvement, ces polarités apparaissent souvent de manière structurée autour du contour de l'objet. Elles peuvent donc aider à vérifier qu'un groupe d'événements ressemble bien au passage d'une balle plutôt qu'à du bruit isolé. La figure 3.5 montre à quoi ressemble cette sortie pour une balle réelle : seuls les pixels dont la luminosité vient de changer produisent un événement, et les deux polarités se répartissent de part et d'autre du contour selon le sens du déplacement.

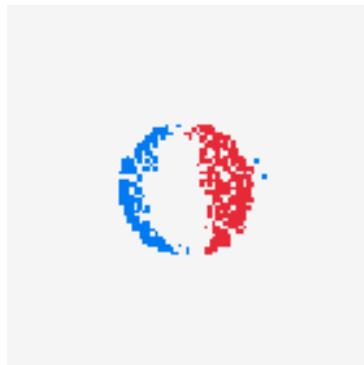


FIGURE 3.5 – Sortie typique de la caméra événementielle pour une balle en mouvement, agrandie autour de la balle. Chaque pixel coloré est un événement : en bleu les événements positifs (augmentation de luminosité), en rouge les événements négatifs (diminution). La balle se déplaçant vers la droite, son bord avant assombrit le fond et génère les événements négatifs, tandis que son bord arrière le découvre et génère les événements positifs.

Ce fonctionnement est particulièrement intéressant pour une balle rapide. La caméra ne produit des données que là où quelque chose change, ce qui réduit fortement la quantité d'information à traiter. En revanche, si l'on accumule trop longtemps les événements, la balle ne forme plus un contour circulaire instantané : elle laisse une trace allongée. L'algorithme doit donc conserver

assez d'événements pour être robuste, mais pas au point de mélanger plusieurs positions de la balle.

3.5.2 Pipeline de l'algorithme

La méthode par *circle fitting* considère que la balle est une sphère dont la projection dans l'image peut être approchée par un cercle. L'idée est de détecter les événements appartenant probablement à la balle, d'ajuster un cercle sur ces points, puis d'utiliser le rayon apparent pour calculer la profondeur. La figure 3.6 résume les étapes principales de cette chaîne de traitement et l'information produite par chacune ; une version détaillée des étapes internes est donnée dans l'annexe B.

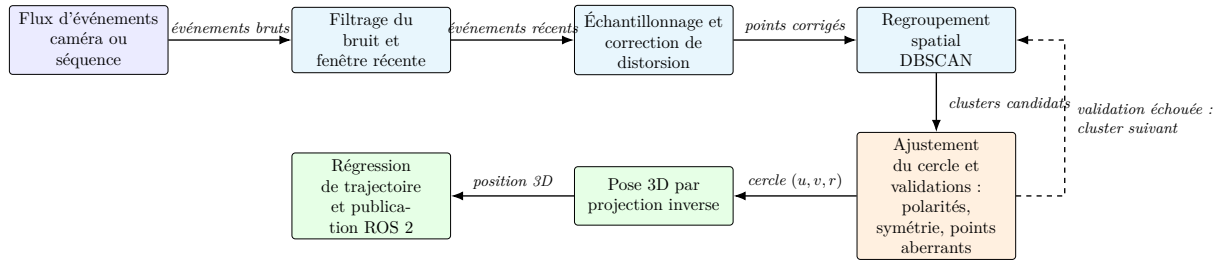


FIGURE 3.6 – Pipeline simplifié de la détection 3D par circle fitting. Chaque flèche porte l'information transmise à l'étape suivante ; lorsqu'une validation échoue, le candidat est rejeté et le cluster suivant est examiné.

Le premier choix important est l'utilisation d'une fenêtre temporelle récente. Cette fenêtre limite les événements analysés aux derniers instants observés par la caméra. Elle réduit le bruit accumulé, évite de mélanger des positions trop anciennes de la balle, et donne une estimation plus réactive. La taille de cette fenêtre est donc un compromis : trop courte, elle ne contient pas assez de points ; trop longue, elle transforme le contour de la balle en traînée.

Après ce fenêtrage, les événements sont échantillonnés. Cette étape ne change pas le modèle géométrique, mais elle est importante pour les performances. DBSCAN doit comparer les points entre eux pour former des groupes denses : si trop d'événements sont conservés, le temps de calcul augmente rapidement. L'échantillonnage garde un sous-ensemble représentatif du flux, ce qui permet de conserver une boucle rapide tout en laissant assez de points pour reconnaître la balle.

L'étape suivante est l'*undistortion*. Les coordonnées image sont corrigées avec la calibration OpenCV [12] afin de compenser la distorsion optique de la caméra. Cette correction est nécessaire avant le clustering final et surtout avant la pose 3D, car le calcul de profondeur suppose que les points suivent le modèle de projection calibré. Dans le pipeline utilisé, l'undistortion est appliquée après l'échantillonnage afin de corriger moins de points et donc de réduire le coût de calcul.

DBSCAN est ensuite utilisé pour regrouper les événements en clusters. Ce choix est adapté au problème car le nombre d'objets visibles n'est pas connu à l'avance et le flux contient du bruit. DBSCAN cherche des zones denses dans l'image, rejette les points isolés, et fournit plusieurs clusters candidats. Chaque cluster peut correspondre à la balle, à une partie de la scène en mouvement, ou à un bruit structuré ; l'étape de circle fitting sert ensuite à sélectionner le candidat le plus cohérent.

3.5.3 Sous-tranches temporelles

Lorsqu'un cluster contient des événements répartis sur une durée trop longue, il peut représenter plusieurs positions successives de la balle. Pour limiter cet effet, l'algorithme peut découper le cluster en sous-tranches. Ces sous-tranches peuvent être définies par fenêtre temporelle ou par nombre d'événements. L'objectif est de retrouver une forme plus proche d'un cercle instantané, au lieu d'ajuster un cercle sur toute la trace. La figure 3.7 illustre une sortie 2D utilisée pour contrôler ce choix.

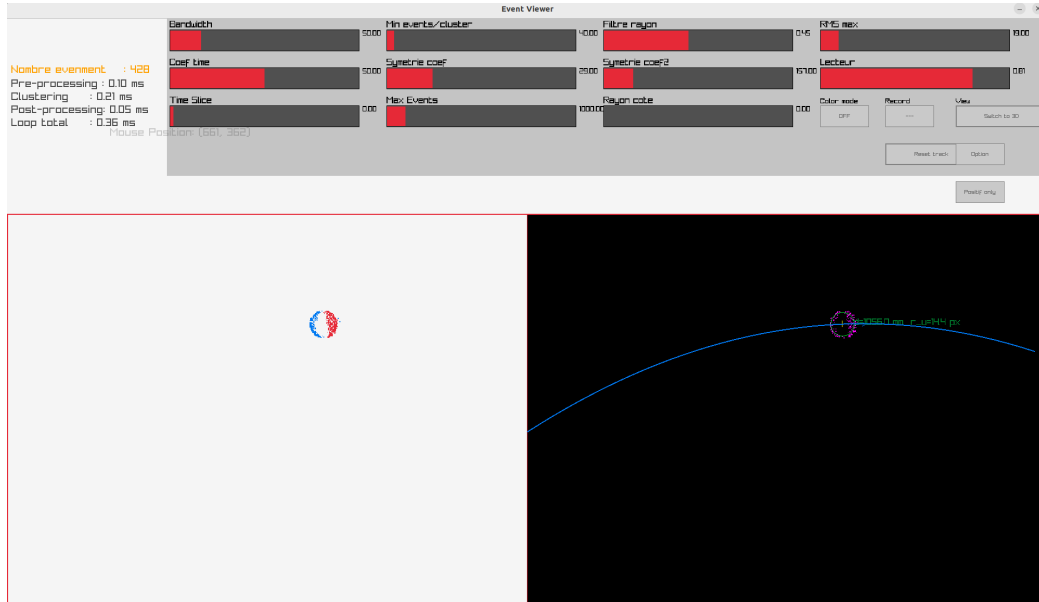


FIGURE 3.7 – Exemple de visualisation 2D utilisée pour contrôler les événements conservés, le cercle détecté et la trajectoire estimée.

Cette étape est particulièrement utile lorsque la balle se déplace vite dans le plan image. Une sous-tranche récente donne une estimation plus proche de la position actuelle, tandis qu'un cluster complet peut être influencé par des événements plus anciens. Elle prépare donc un nuage de points plus stable pour l'ajustement du cercle.

3.5.4 Ajustement du cercle et validation

Le cercle estimé fournit trois informations principales :

$$(u, v, r),$$

où u et v sont les coordonnées du centre dans l'image et r est le rayon apparent en pixels. L'algorithme effectue deux ajustements successifs. Le premier cercle est calculé sur le cluster candidat afin d'obtenir une estimation initiale du centre et du rayon. Cette estimation sert ensuite à analyser les polarités, calculer le résidu RMS et éliminer les points aberrants.

La validation par symétrie est une étape importante de première identification de la balle. Après le premier cercle, le centre estimé sert de référence pour regarder comment les événements positifs et négatifs sont répartis autour du candidat. Pour chaque événement p_i , on calcule le vecteur radial normalisé :

$$d_i = \frac{p_i - c}{\|p_i - c\|},$$

où c est le centre du cercle estimé. Les vecteurs sont ensuite séparés selon leur polarité. L'algorithme accumule les directions des événements positifs et négatifs :

$$D_+ = \sum_{p_i=+} d_i, \quad D_- = \sum_{p_i=-} d_i.$$

Un vrai passage de balle produit généralement deux groupes de polarités opposés : un bord de la balle provoque une augmentation de luminosité et l'autre une diminution. L'algorithme vérifie donc d'abord que les deux polarités sont présentes et que leurs nombres restent relativement équilibrés :

$$\text{sym} = \frac{|N_+ - N_-|}{N_+ + N_-}.$$

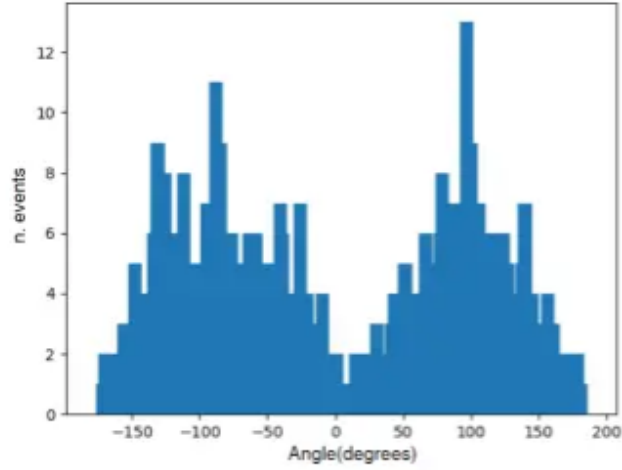
Si cette valeur est trop grande, le cluster est rejeté car il ressemble davantage à du bruit ou à un objet non symétrique qu'à une balle. Ensuite, l'algorithme calcule l'angle moyen de chaque polarité :

$$\theta_+ = \text{atan2}(D_{+,y}, D_{+,x}), \quad \theta_- = \text{atan2}(D_{-,y}, D_{-,x}).$$

La séparation entre ces deux directions doit être proche de π , c'est-à-dire que les deux groupes doivent être approximativement opposés :

$$||\theta_+ - \theta_-| - \pi| < \varepsilon.$$

Cette étape ne donne pas encore la pose 3D finale, mais elle permet d'écarter tôt les clusters qui ne possèdent pas la structure attendue d'une balle en mouvement. Elle rend donc le choix du cluster plus robuste avant le filtrage des points aberrants et le second ajustement de cercle. La figure 3.8, issue du rapport de stage d'Antonio [1], sert ici de schéma explicatif pour visualiser cette séparation des polarités.



(d) Distribution of all events over different angles.

FIGURE 3.8 – Schéma explicatif issu du rapport de stage d’Antonio [1] : les événements positifs et négatifs doivent former deux directions moyennes opposées autour du centre du cercle candidat.

Le second cercle est recalculé après ce nettoyage. Cette deuxième régression est plus fiable car elle s’appuie sur un nuage de points déjà filtré. Le but n’est pas de complexifier inutilement l’algorithme, mais d’éviter qu’un petit nombre de points isolés élargisse artificiellement le rayon. Cette précaution est importante, car la profondeur dépend directement de ce rayon apparent. La figure 3.9 montre la sortie 2D de l’algorithme après clustering, validation et estimation du cercle.



FIGURE 3.9 – Sortie 2D de l’algorithme. La partie gauche montre les événements bruts, tandis que la partie droite montre les clusters, le cercle retenu et la trajectoire image. La flèche verte indique l’estimation de la direction de la balle ; cette direction est stabilisée par la validation de polarité et de symétrie du cluster.

3.5.5 Détermination de la pose 3D

En connaissant le rayon réel R de la balle et la focale f_x , la profondeur est estimée à partir du rayon apparent :

$$Z = \frac{f_x R}{r}.$$

La position 3D complète est ensuite calculée par projection inverse :

$$X = \frac{u - c_x}{f_x} Z, \quad Y = \frac{v - c_y}{f_y} Z.$$

Dans le projet, la balle utilisée dans les séquences simulées possède un rayon de 20 mm. Le même ordre de grandeur est utilisé pour l'estimation réelle. La pose obtenue est exprimée en millimètres dans le repère caméra, puis convertie en mètres pour l'affichage 3D.

La trajectoire représentée sur la figure 3.10 correspond à de bonnes conditions de détection : le cluster de la balle est bien isolé, le cercle ajusté reste cohérent avec la projection de la balle et le rayon apparent n'est pas trop perturbé par une traînée événementielle. Elle illustre donc le comportement attendu de la méthode lorsque le signal respecte bien l'hypothèse circulaire.

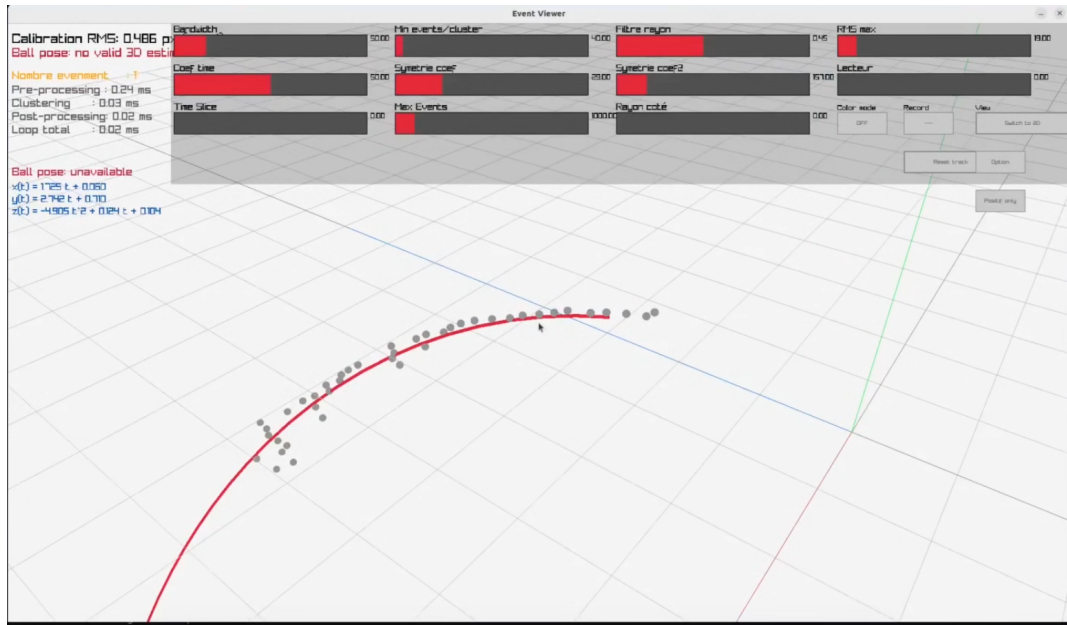


FIGURE 3.10 – Estimation 3D obtenue dans de bonnes conditions de détection, à partir du centre et du rayon du cercle détecté dans l'image.

3.5.6 Régression de trajectoire 3D

Chaque position 3D valide est ajoutée à l'historique. La trajectoire est ensuite modélisée par des régressions simples :

$$X(t) = a_x t + b_x, \quad Y(t) = a_y t + b_y, \quad Z(t) = a_z t^2 + b_z t + c_z.$$

Les coefficients de ces modèles sont estimés par moindres carrés. Les régressions ne sont pas résolues par un système matriciel général recalculé à chaque itération : comme la forme des modèles est connue à l'avance, les sommes nécessaires ($\sum t_i$, $\sum t_i^2$, $\sum t_i q_i$, etc.) sont accumulées au fil des mesures, et les coefficients s'en déduisent directement avec peu d'opérations. Ce choix garde la boucle de traitement rapide, ce qui est essentiel pour une application temps réel. Les expressions complètes des coefficients sont données dans l'annexe A.

Une version pondérée peut aussi être activée. Elle garde les mêmes modèles, mais remplace les sommes simples par des sommes pondérées. Le poids d'un point combine un terme de récence

et un terme robuste basé sur le résidu :

$$w_i = \exp\left(-3 \frac{t_{\max} - t_i}{t_{\max} - t_{\min}}\right) \cdot \frac{1}{1 + \left(\frac{r_i}{s}\right)^2},$$

où r_i est le résidu par rapport à la première régression et s est une échelle robuste calculée à partir de la médiane des résidus. Les points récents et cohérents ont donc plus d'influence que les anciens points ou les mesures aberrantes.

Ce choix correspond au mouvement attendu d'un projectile : deux axes sont modélisés de manière linéaire, tandis que l'axe soumis principalement à la gravité est modélisé par une parabole. L'objectif n'est pas seulement d'afficher les points mesurés ; la trajectoire sert surtout de filtre temporel pour stabiliser la suite de positions 3D et vérifier que les poses successives restent cohérentes.

La position de la balle est publiée en continu dans ROS 2 [11] à partir des poses 3D validées. La trajectoire 3D sert donc principalement à lisser et filtrer la perception, plutôt qu'à remplacer la mesure instantanée par une prédiction longue.

3.5.7 Limite principale : sensibilité du rayon

La faiblesse de cette méthode vient du fait que toute la profondeur dépend du rayon apparent r . Une petite erreur sur r produit une grande erreur sur Z , surtout lorsque la balle est éloignée et que son rayon en pixels est faible.

À partir de :

$$Z = \frac{f_x R}{r},$$

on obtient la sensibilité :

$$\frac{dZ}{dr} = -\frac{f_x R}{r^2} = -\frac{Z}{r}.$$

Cela signifie qu'une erreur de 1 px sur le rayon produit environ Z/r d'erreur sur la profondeur.

TABLE 3.2 – Sensibilité de la profondeur pour la caméra réelle, avec $f_x \approx 761$ px et $R = 20$ mm.

Z	r image	Largeur	Erreur 1 px rayon	Erreur 1 px largeur
1.00 m	15.23 px	30.45 px	6.6 cm	3.3 cm
1.50 m	10.15 px	20.31 px	14.8 cm	7.4 cm
2.00 m	7.61 px	15.23 px	26.3 cm	13.1 cm

À 1.50 m, le rayon apparent vaut seulement environ 10.15 px. Une erreur d'un pixel sur ce rayon représente donc environ 15 cm d'erreur de profondeur. Si l'on raisonne sur une largeur ou un diamètre, l'erreur d'un pixel correspond à environ 7 cm. Cette sensibilité explique pourquoi le circle fitting devient insuffisant pour une balle rapide.

En pratique, la caméra événementielle ne voit pas toujours un contour circulaire net. Lors d'un mouvement rapide, les événements s'étalent dans le temps et forment une traînée. Le cercle

ajusté peut alors surestimer le rayon apparent. Comme Z est inversement proportionnel à r , un rayon surestimé donne une balle artificiellement trop proche.

3.5.8 Décalage du plan estimé à haute vitesse

Un effet visible apparaît lorsque l'on lance une balle de ping-pong dans un plan bien défini devant la caméra. Physiquement, la trajectoire devrait appartenir à un plan réaliste, approximativement perpendiculaire au sol. En vue de dessus, ce plan peut être représenté par une droite. Or les estimations 3D issues du circle fitting ne restent pas toujours sur cette droite : elles peuvent être décalées vers une profondeur plus faible. La figure 3.11 donne deux exemples de ce décalage.

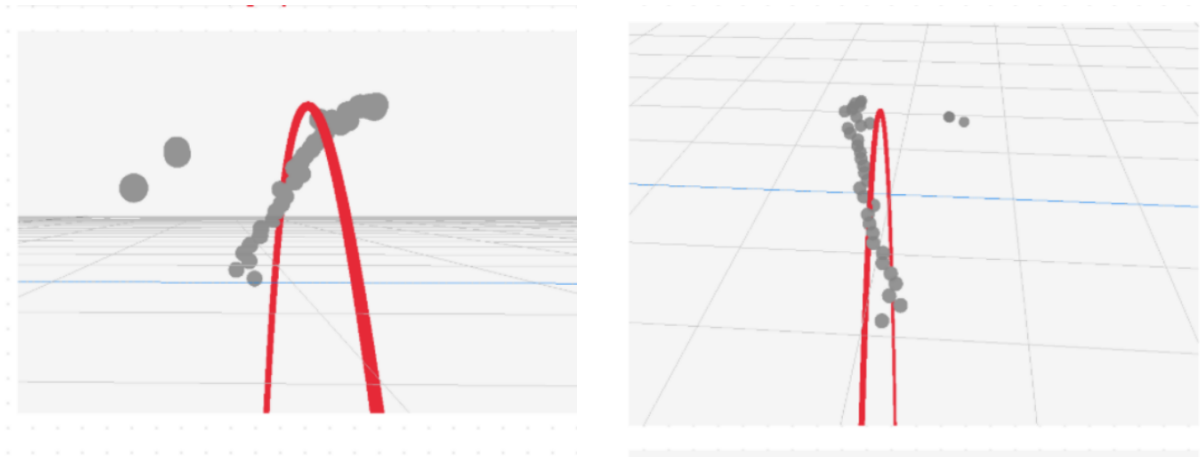


FIGURE 3.11 – Exemples de décalage de la trajectoire 3D estimée lorsque la balle est rapide. Les points estimés ne suivent pas toujours le plan vertical attendu pour le lancer.

La cause principale est la même que précédemment : à haute vitesse, la traînée événementielle devient plus grande. Le cercle ajusté peut alors englober une zone trop large, ce qui augmente le rayon apparent r . Comme la profondeur est calculée avec $Z = f_x R / r$, une surestimation de r produit directement une profondeur trop faible. Les points 3D sont alors ramenés artificiellement vers la caméra, ce qui décale le plan estimé par rapport au plan réel du mouvement.

Pour vérifier si cette erreur peut être corrigée, on peut chercher une relation entre la vitesse de la balle et l'erreur de profondeur. Dans l'outil de visualisation, la vue de dessus ajuste d'abord une droite aux estimations 3D projetées dans le plan X/Y :

$$Y = aX + b.$$

Cette droite représente le plan vertical attendu vu du dessus. Pour chaque estimation $P_i = (X_i, Y_i, Z_i)$, l'outil calcule ensuite la distance orthogonale à cette droite :

$$d_i = \frac{|aX_i - Y_i + b|}{\sqrt{a^2 + 1}}.$$

La vitesse utilisée pour comparer cette erreur est la vitesse dans le plan caméra X/Z , calculée à partir de la régression 3D :

$$v_{XZ}(t) = \sqrt{a_X^2 + (2a_Z t + b_Z)^2}.$$

La distance d_i est alors tracée en fonction de $v_{XZ}(t_i)$, puis une régression linéaire est ajustée :

$$d \approx \alpha v_{XZ} + \beta.$$

Si une proportionnalité apparaît, cela indique que le biais de profondeur dépend de la vitesse. Cette relation pourrait ensuite servir à corriger soit le rayon estimé, soit directement la profondeur, par exemple en ajoutant une correction dépendante de la vitesse. La figure 3.12 présente cette analyse dans l'outil de visualisation.

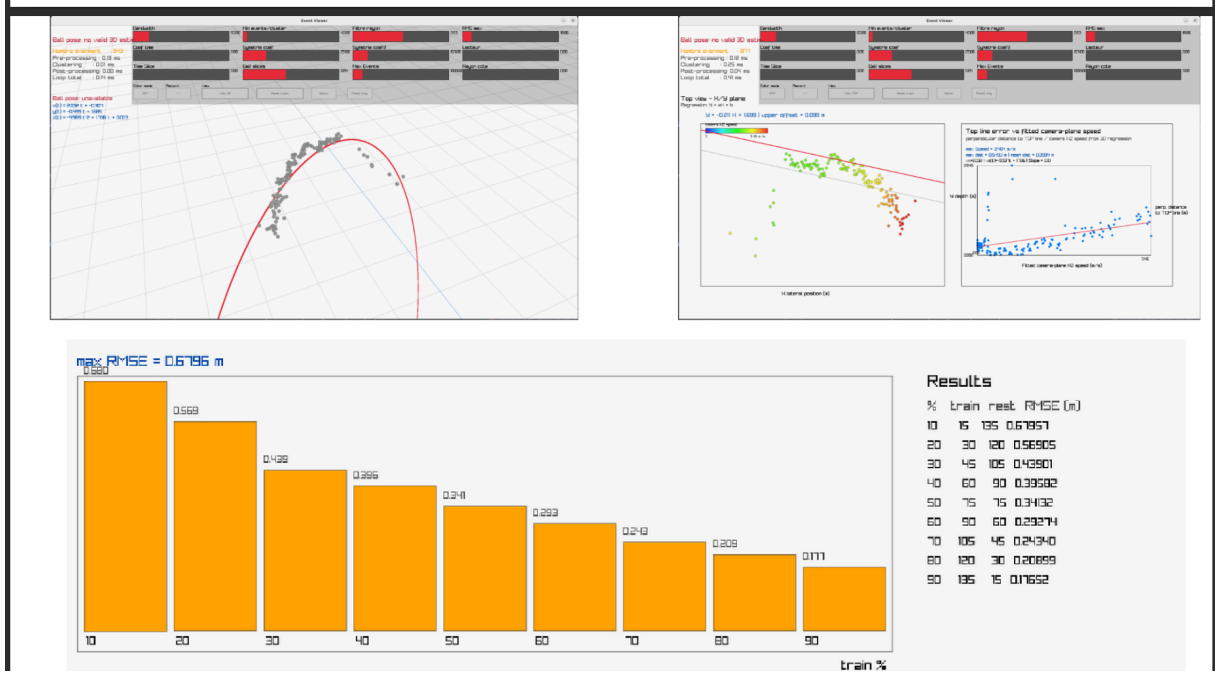


FIGURE 3.12 – Analyse du biais de profondeur. En haut à droite, la vue de dessus montre le décalage des estimations par rapport au plan attendu ; le graphique adjacent trace la distance orthogonale à ce plan en fonction de la vitesse estimée dans le plan caméra. La tendance linéaire suggère une proportionnalité entre vitesse et erreur de profondeur.

La partie basse de cette figure correspond à la vue RMSE de l'outil de visualisation. Pour chaque pourcentage p entre 10% et 90%, l'algorithme utilise les $p\%$ premiers points 3D pour ajuster une trajectoire :

$$X(t) = a_X t + b_X, \quad Y(t) = a_Y t + b_Y, \quad Z(t) = a_Z t^2 + b_Z t + c_Z.$$

Il teste ensuite cette trajectoire sur les points restants, c'est-à-dire les $(100 - p)\%$ derniers points. Si N_p est le nombre de points utilisés pour l'apprentissage, la mesure affichée est :

$$\text{RMSE}(p) = \sqrt{\frac{1}{N - N_p} \sum_{i=N_p}^{N-1} \|\hat{P}_i^{(p)} - P_i\|^2}.$$

Cette RMSE ne compare pas directement l'estimation à une vérité terrain externe. Elle mesure plutôt la cohérence interne de la trajectoire estimée : on regarde si les premiers points permettent de prédire correctement la suite des estimations. Lorsque le biais de profondeur varie avec la

vitesse, cette erreur augmente car la trajectoire estimée devient moins compatible avec un modèle balistique simple.

3.5.9 Conclusion sur la méthode par circle fitting

La méthode par circle fitting donne une base simple et efficace lorsque la balle produit un contour événementiel suffisamment circulaire. Elle permet d'obtenir rapidement une pose 3D à partir de peu d'informations : un centre image, un rayon apparent et la calibration caméra. Cependant, sa limite principale est structurelle : toute la profondeur dépend directement du rayon estimé. Or ce rayon est lui-même très sensible au bruit, à la densité d'événements, au choix de la fenêtre temporelle et surtout à la vitesse de la balle.

Lorsque la balle se déplace vite, la traînée événementielle augmente le rayon apparent. La profondeur calculée devient alors trop faible, ce qui décale la trajectoire 3D. On peut essayer de corriger ce biais, par exemple en cherchant une relation entre vitesse et erreur de profondeur, puis en appliquant une compensation. Cette piste est possible, mais elle devient rapidement laborieuse : il faut estimer un coefficient fiable, vérifier qu'il reste valable sur plusieurs lancers, et tenir compte de facteurs difficiles à contrôler.

Parmi ces facteurs, on peut citer la luminosité de la scène, le contraste de la balle, l'angle de tir, la distance à la caméra, la rotation éventuelle de la balle, la densité d'événements produite par la caméra et les erreurs de calibration. Certains de ces effets sont difficiles à mesurer directement, et d'autres peuvent varier d'une séquence à l'autre. Corriger uniquement le rayon risque donc de produire une solution fragile, très dépendante des conditions expérimentales.

Il est donc plus intéressant de chercher une méthode moins sensible à une erreur locale sur le rayon. C'est la motivation de la méthode Trace. Au lieu de forcer un cercle sur une forme parfois allongée, elle exploite la forme réellement observée dans le flux événementiel. La largeur de la trace est estimée à partir de plusieurs coupes et de nombreux événements, ce qui réduit l'influence d'un point isolé ou d'un cercle mal ajusté. Cette approche est plus prometteuse pour les balles rapides, car elle utilise directement l'information produite par la caméra événementielle au lieu de la ramener à tout prix à un contour circulaire instantané.

3.6 Méthode par trace événementielle

3.6.1 Motivation

La méthode par trace part d'une observation simple : pour une balle rapide, l'information utile n'est pas seulement contenue dans un cercle instantané. Elle est répartie dans toute la trace événementielle produite pendant le déplacement de la balle. Cette trace contient beaucoup plus de points qu'un seul contour local, et elle correspond mieux à ce que la caméra événementielle observe réellement lorsque la balle traverse vite le champ de vision.

L'idée est donc de mesurer la largeur réelle de la trace, puis d'utiliser cette largeur comme diamètre apparent de la balle. La profondeur n'est plus déduite d'un rayon de cercle ajusté sur quelques événements, mais d'un ruban estimé à partir de nombreuses coupes locales. Cette approche est plus robuste face aux points isolés, aux contours incomplets et aux déformations créées par la vitesse. La figure 3.13 montre la trace test utilisée pour valider ce principe.

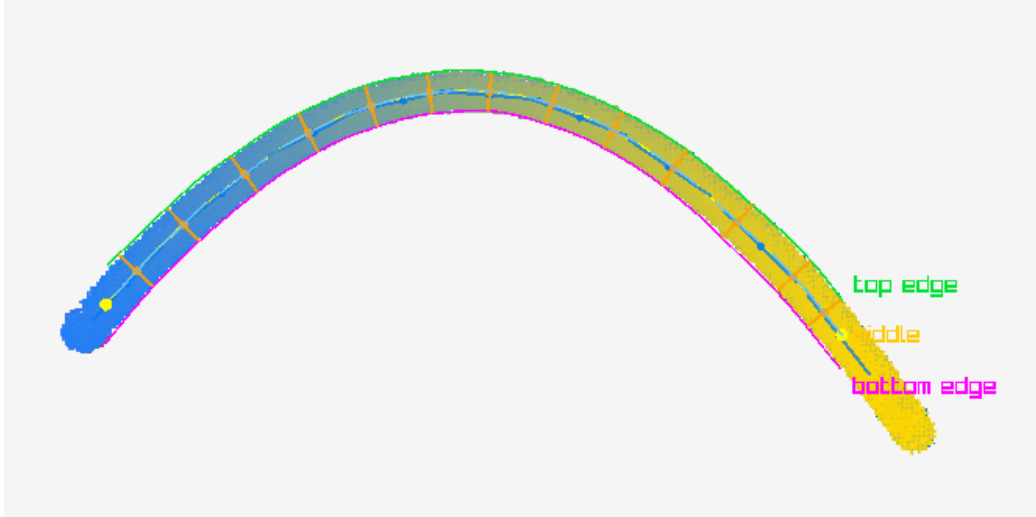


FIGURE 3.13 – Trace test utilisée pour valider la méthode. Les événements forment un ruban courbe : le bord haut est en vert, la ligne médiane en jaune, le bord bas en magenta, et les segments orange représentent les mesures locales de largeur.

3.6.2 Reconstruction du ruban

La reconstruction transforme le nuage d'événements en un ruban dont la largeur peut être mesurée. Elle suit cinq étapes principales ; les critères précis et les expressions utilisées à chaque étape sont détaillés dans l'annexe C.

1. *Sélection des événements.* Une fenêtre mobile suit la trajectoire 2D déjà estimée et accumule, sur une courte mémoire temporelle, les événements proches de la courbe prédite. La trace analysée ne contient ainsi, en principe, que la balle.
2. *Repère local par PCA.* Le centre du nuage est estimé de manière robuste, puis une analyse en composantes principales (PCA) fournit la direction principale de la trace, corrigée sur des sous-parties temporelles lorsque la trajectoire est courbe. Chaque événement est alors projeté dans ce repère et repéré par deux coordonnées : s , sa position le long de la direction principale, et h , sa position sur la perpendiculaire à cette direction, c'est-à-dire dans l'épaisseur du ruban. Cette transformation convertit un problème 2D difficile en une série de problèmes 1D.
3. *Découpage en bins spatiaux.* La trace est découpée en petites portions le long de s . Dans chaque bin, il ne reste plus qu'un problème 1D : les valeurs h des événements du bin forment un paquet sur l'axe perpendiculaire, dont il faut trouver les deux extrémités. Chaque bin fournit ainsi une estimation locale unique : la position du bord bas, celle du bord haut, et la ligne médiane qui en est déduite. La largeur locale du ruban est la distance entre les deux bords, mesurée perpendiculairement à la direction de la trace.
4. *Détection de bords soutenus.* Pour trouver ces extrémités, prendre simplement le minimum et le maximum des valeurs h serait trop sensible au bruit : un seul événement isolé deviendrait un faux bord et fausserait la largeur, donc la profondeur. L'axe h est donc balayé depuis chaque extrémité, et un bord n'est accepté que s'il est soutenu par suffisamment d'événements voisins.
5. *Filtrage et courbes du ruban.* Les bins incohérents avec la tendance générale du ruban ou avec leurs voisins sont rejetés. Les trois courbes du ruban, bord bas, ligne médiane et bord haut, sont ensuite ajustées sur les échantillons restants par des régressions locales pondérées, ce qui garde le ruban régulier sans lui imposer une forme globale unique.

La figure 3.14 montre le résultat de cette reconstruction dans l'interface de visualisation.



FIGURE 3.14 – Vue Trace dans l'interface pour une balle lancée vers la caméra. Les événements récents sont colorés selon le temps, puis le ruban est reconstruit avec un bord haut, une ligne médiane, un bord bas et plusieurs mesures de largeur.

3.6.3 Conversion 3D par largeur de trace

La conversion 3D repose sur le même principe géométrique que le circle fitting, mais la grandeur mesurée n'est plus le rayon d'un cercle. Pour plusieurs instants de la trace, la méthode mesure une largeur perpendiculaire à la ligne médiane. Cette largeur est utilisée comme diamètre apparent de la balle :

$$Z = \frac{f_{\text{eff}} D}{w_{px}},$$

où D est le diamètre réel de la balle et w_{px} la largeur mesurée en pixels. Comme la largeur n'est pas forcément horizontale ou verticale dans l'image, la méthode utilise une focale effective dans la direction de mesure :

$$f_{\text{eff}} = \sqrt{(f_x n_x)^2 + (f_y n_y)^2}.$$

Le centre image (u, v) de la mesure permet ensuite de retrouver les coordonnées caméra :

$$X = \frac{(u - c_x)Z}{f_x}, \quad Y = \frac{(v - c_y)Z}{f_y}.$$

Dans l'affichage 3D, ces coordonnées sont ensuite remappées dans le repère utilisé par l'interface. Les positions 3D aberrantes sont filtrées par sauts temporels et par résidu de trajectoire. Le

résultat final est donc une suite de points 3D, puis une trajectoire ajustée sur ces points. Les sorties obtenues avec cette conversion sont présentées dans la section 3.7, où la simulation fournit une vérité terrain pour comparer directement l'estimation à la position réelle de la balle.

3.6.4 Régression pondérée de trajectoire

Une régression est ensuite calculée sur les positions 3D obtenues. Les modèles et la pondération sont les mêmes que ceux présentés pour la méthode par circle fitting : axes horizontaux linéaires, axe vertical quadratique, et poids combinant un terme de récurrence et un terme robuste basé sur le résidu. Sans pondération, une mauvaise estimation ancienne peut tirer la courbe finale et créer un décalage visible. Avec pondération, la trajectoire suit davantage les points récents tout en diminuant l'effet des mesures aberrantes. La comparaison visuelle entre les deux réglages est présentée dans la section de validation.

3.6.5 Bilan de la méthode Trace

La méthode Trace est plus prometteuse que le circle fitting pour les tirs rapides, car elle exploite la forme réellement produite par la balle dans le flux événementiel. Au lieu de dépendre fortement d'un rayon instantané, elle mesure une largeur sur plusieurs coupes de la trace et combine ces mesures dans le temps. Elle reste sensible aux erreurs de largeur, à la qualité de la fenêtre de suivi et à la quantité d'événements disponibles, mais son information de départ est mieux adaptée au signal observé par la caméra événementielle.

3.7 Simulation et validation

3.7.1 Séquences simulées

Pour tester l'algorithme dans un cadre contrôlé, des séquences simulées ont été générées avec Isaac Sim [14]. Une balle est lancée devant une caméra virtuelle dont les paramètres intrinsèques sont les mêmes que ceux utilisés par l'algorithme. La sortie vidéo de cette caméra est ensuite convertie en événements avec `v2e` [16, 17], afin d'obtenir un flux proche de celui fourni par une caméra événementielle. En parallèle, Isaac Sim [14] fournit la position réelle de la balle à chaque instant.

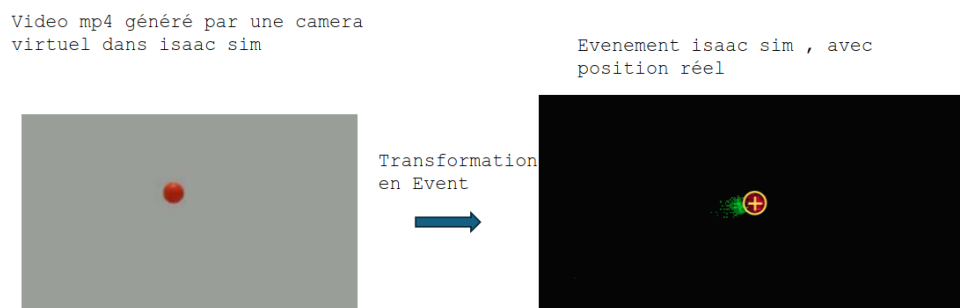


FIGURE 3.15 – Passage d'une vidéo générée en simulation vers un flux événementiel avec `v2e` [16, 17].

Cette organisation permet de comparer directement les positions estimées par l'algorithme avec une vérité terrain. Dans les figures 3D de cette section, chaque carré de la grille représente 25 cm. Les estimations de l'algorithme sont affichées en bleu, tandis que les positions réelles de référence sont affichées en rouge.

Les paramètres complets de génération de ces séquences (résolution, fréquence d'entrée, seuils de contraste et résolution temporelle des événements) sont donnés dans l'annexe D. Les trois séquences retenues sont récapitulées dans le tableau 3.3.

TABLE 3.3 – Caractéristiques des séquences simulées utilisées pour la validation.

Séquence	Durée	Événements bruts	Événements filtrés
sequence_0001	2.0 s	49 232	38 981
sequence_0002	2.0 s	47 835	37 909
sequence_0003	0.468 s	28 016	22 478

La vérité terrain contient la position réelle de la balle dans le repère caméra, ainsi que sa projection image. Elle permet de comparer la position estimée par l'algorithme avec la trajectoire attendue. Cette comparaison est importante, car une trajectoire peut sembler visuellement cohérente tout en ayant une erreur de profondeur trop élevée pour l'interception robotique.

3.7.2 Résultats 3D de la méthode Trace

La figure 3.16 montre les sorties 3D obtenues avec la méthode Trace sur une séquence simulée. Les points bleus clairs correspondent aux mesures de largeur converties en positions 3D. La courbe bleue est la trajectoire estimée à partir de ces points. Les points rouges représentent la trajectoire réelle fournie par Isaac Sim [14].

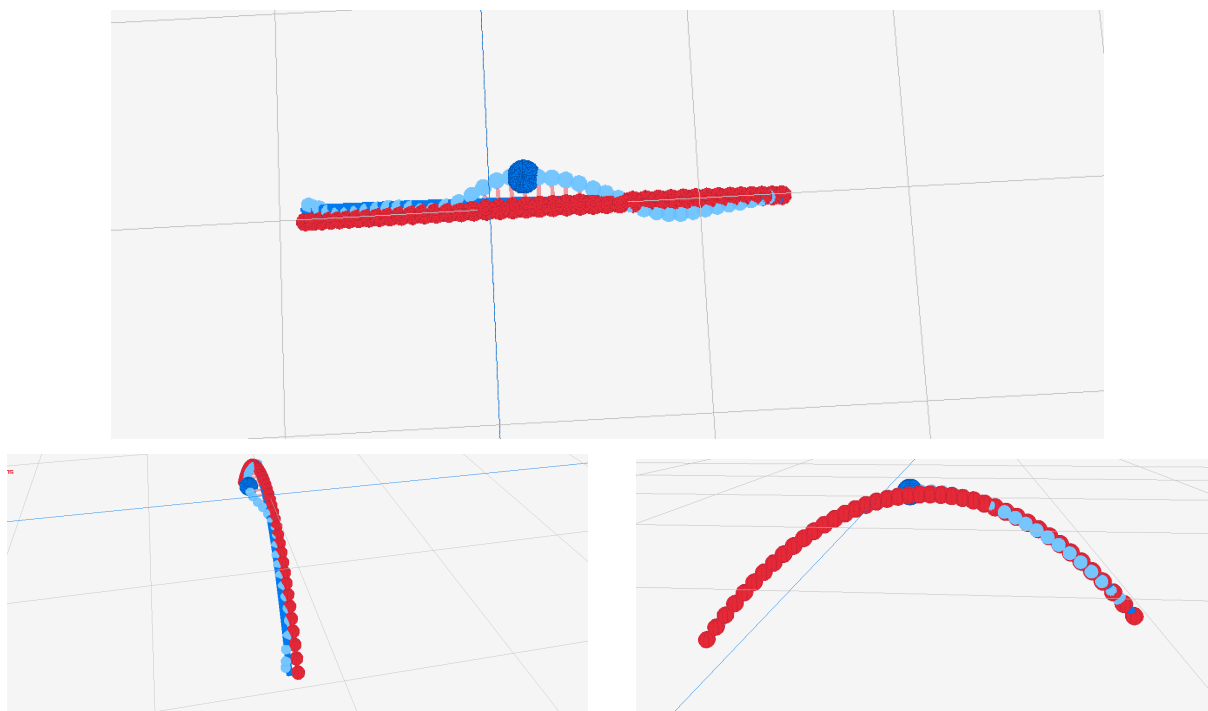


FIGURE 3.16 – Sorties 3D obtenues avec la méthode Trace. Chaque carré de la grille correspond à 25 cm. Les points bleus sont les estimations de l’algorithme et les points rouges sont les positions réelles issues d’Isaac Sim [14].

La figure 3.17 détaille l’évolution de cette trajectoire lorsque de nouvelles estimations sont ajoutées au fil du temps.

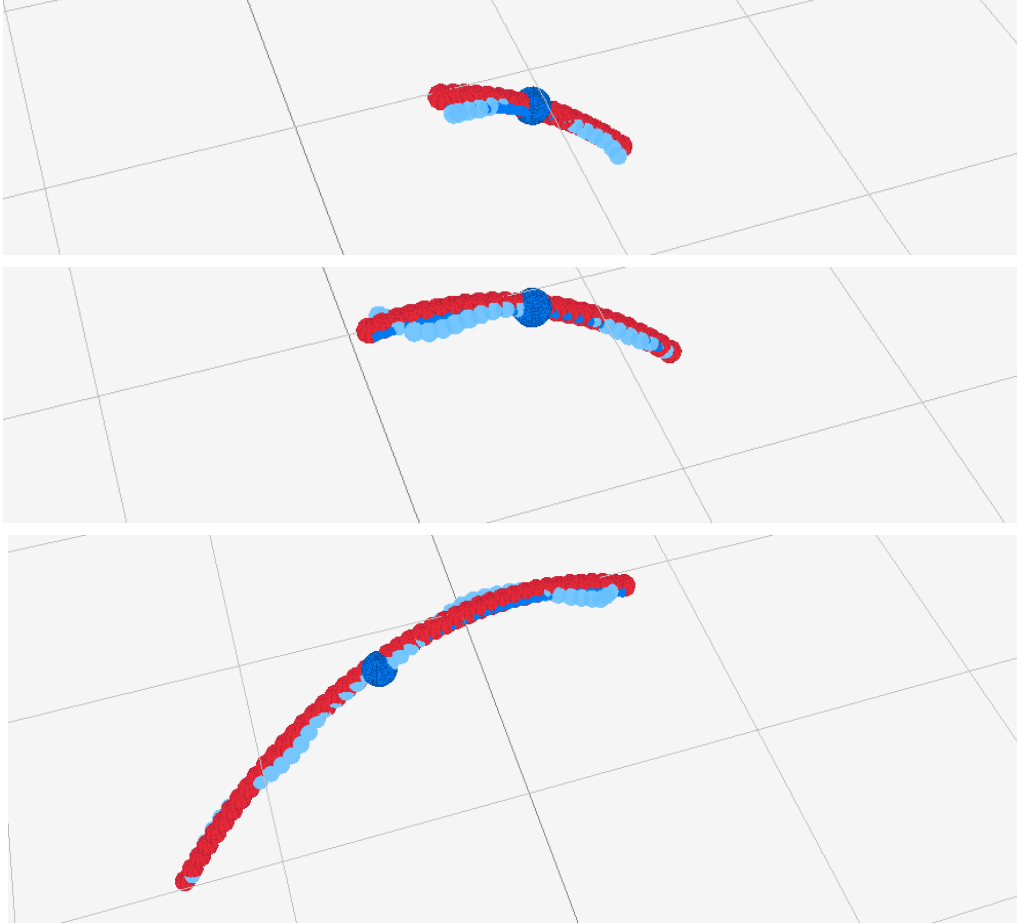


FIGURE 3.17 – Évolution de la trajectoire Trace dans le temps. À mesure que de nouvelles largeurs sont mesurées, de nouveaux points 3D sont ajoutés et la courbe estimée se stabilise.

3.7.3 Effet de la régression pondérée

La régression pondérée est évaluée sur la même sortie 3D. Sans poids ajustés, la trajectoire bleue peut être attirée par des points anciens ou par des estimations moins fiables. Avec les poids de récence et de robustesse, la courbe tient davantage compte des points récents et réduit l'influence des mesures aberrantes. L'objectif n'est pas de changer le modèle de trajectoire, mais de rendre son estimation plus exploitable en temps réel. La figure 3.18 compare visuellement ces deux réglages.

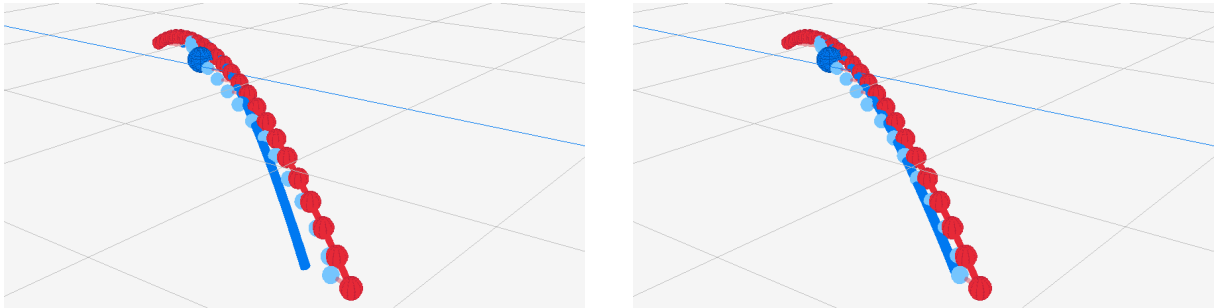


FIGURE 3.18 – Comparaison entre la régression Trace sans poids ajustés à gauche et avec poids ajustés à droite, pour une balle lancée vers la caméra. La pondération récente et robuste aligne mieux la courbe bleue sur les estimations utiles de la trajectoire.

3.7.4 Rôle des tests

Les séquences simulées servent à séparer deux problèmes :

- la qualité de l'algorithme géométrique ;
- les difficultés liées au matériel réel, à l'éclairage et au bruit de la caméra.

Lorsque l'algorithme échoue sur une séquence simulée avec vérité terrain, l'erreur vient principalement du traitement ou du modèle géométrique. Lorsqu'il fonctionne en simulation mais devient instable sur la caméra réelle, il faut plutôt analyser la calibration, la polarité des événements, la densité de la trace et les conditions expérimentales.

Chapitre 4

Intégration robotique et contrôle

4.1 Calibration extrinsèque par mire événementielle

4.1.1 Intuition générale

Les méthodes de perception précédentes estiment la balle dans le repère de la caméra. Pour qu'un robot puisse utiliser cette information, il faut exprimer cette position dans le repère de sa base. Le problème de calibration extrinsèque consiste donc à estimer une transformation fixe entre la caméra événementielle et la base du robot :

$${}^B T_C.$$

Le principe retenu est d'utiliser le robot lui-même comme système de déplacement connu. Une mire dynamique est affichée sur un écran fixé au bout du bras robotique. Comme l'écran est solidaire de l'effecteur, sa pose dans la base du robot peut être calculée à partir de la cinématique directe et de la géométrie du support mécanique. La caméra événementielle observe cette mire pour plusieurs poses du robot. Chaque observation donne une contrainte géométrique, et l'ensemble des poses permet d'estimer la transformation fixe entre la caméra et le robot. La mire utilisée est celle déjà développée pour la calibration intrinsèque (section 3.4, figure 3.3).

Cette approche est intéressante car elle ne demande pas de déplacer manuellement une mire dans l'espace. Le robot fournit directement des poses connues et répétables, ce qui rend possible une calibration multi-poses cohérente.

4.1.2 Montage envisagé

Le montage repose sur quatre éléments principaux :

- une caméra événementielle DVXplorer [9] fixe par rapport à l'environnement ;
- un robot 6 DOF dont la cinématique directe fournit la pose de l'effecteur ;
- un support imprimé en 3D reliant l'effecteur à un écran ;
- une mire lumineuse affichée sur cet écran.

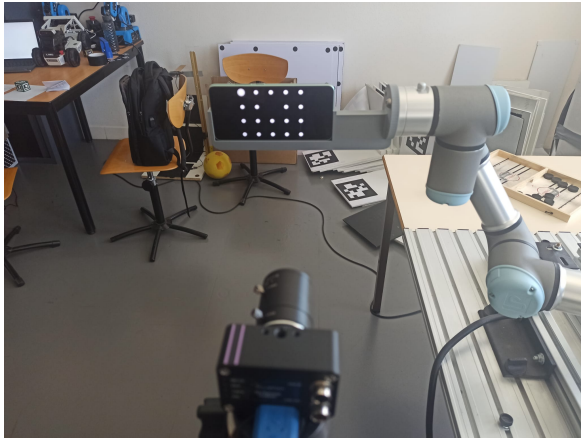


FIGURE 4.1 – Smartphone installé sur le support utilisé pour afficher la mire dynamique. À gauche, vue de face du montage ; à droite, vue de côté montrant le placement du téléphone sur le support.

L'écran peut être un smartphone, une tablette légère ou un petit écran dédié. Un écran trop lourd est à éviter, car il peut provoquer une flexion du support ou de l'effecteur et fausser la pose réelle de la mire. La rigidité mécanique est donc aussi importante que la précision algorithmique : si la transformation entre l'effecteur et l'écran varie pendant le mouvement, la calibration cherchera à expliquer une erreur qui ne vient pas de la caméra.

4.1.3 Repères et transformations

On utilise les repères suivants :

- B : repère de la base du robot ;
- E : repère de l'effecteur, de la flange ou du TCP robot ;
- S : repère de la mire affichée sur l'écran ;
- C : repère de la caméra événementielle.

Pour une pose robot i , la cinématique directe donne la transformation ${}^B T_{E,i}$. La transformation ${}^E T_S$ décrit le placement mécanique de la mire par rapport à l'effecteur. Dans le montage envisagé, elle provient uniquement de la CAO du support imprimé en 3D. Le support est conçu de telle sorte que le smartphone soit maintenu sans jeu et que sa position par rapport à l'effecteur reste fixe pendant les acquisitions.

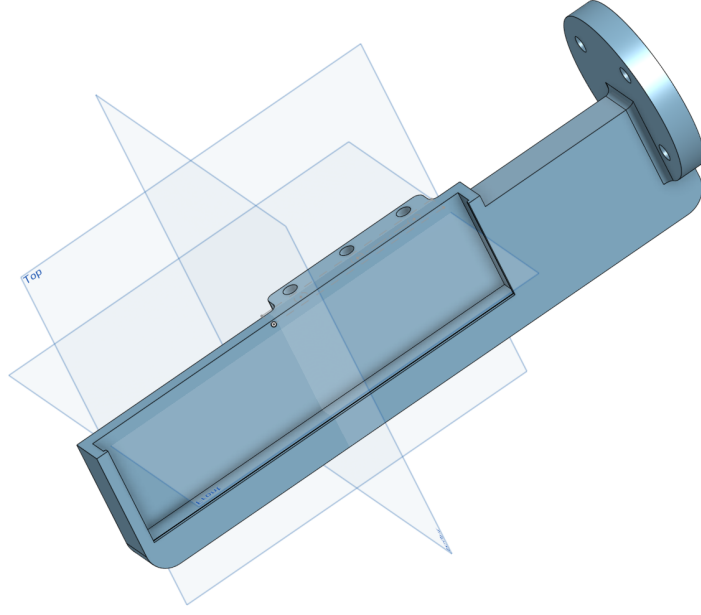


FIGURE 4.2 – Modèle CAO du support utilisé pour définir la position de la mire par rapport à l’effecteur.

La position réelle de l’écran dans le téléphone doit aussi être prise en compte, car la mire n’occupe pas toute la surface physique du smartphone. Cette position est estimée à partir d’une image du téléphone : on mesure en pixels l’épaisseur des bords entre le contour du téléphone et la zone active de l’écran, puis on convertit ces valeurs en millimètres par proportionnalité, en utilisant les dimensions réelles du téléphone comme référence. Cette méthode revient à appliquer une règle de trois entre les dimensions mesurées sur l’image et les dimensions physiques ; les valeurs mesurées sont détaillées dans l’annexe E.

La pose de la mire dans la base du robot est alors :

$${}^B T_{S,i} = {}^B T_{E,i} {}^E T_S.$$

De son côté, la caméra observe la mire et permet d’estimer la transformation ${}^C T_{S,i}$, c’est-à-dire la pose de la mire dans le repère caméra. Pour une pose isolée, on peut en déduire une première estimation de la pose caméra dans la base :

$${}^B T_{C,i} = {}^B T_{S,i} \left({}^C T_{S,i} \right)^{-1}.$$

Une seule pose reste cependant fragile : une erreur de détection, un mauvais angle de vue ou une légère erreur mécanique peut produire une estimation biaisée. L’objectif réel est donc d’utiliser plusieurs poses et de trouver une transformation unique ${}^B T_C$ qui explique au mieux toutes les observations.

4.1.4 Mire événementielle et détection des points

La mire affichée sur l’écran et sa détection dans le flux d’événements sont exactement celles présentées pour la calibration intrinsèque, dans la section 3.4 : une grille asymétrique de disques

clignotants, détectée par image d'activité, seuillage, regroupement en composantes connexes, puis barycentre pondéré. Le disque manquant et le disque-ancree y jouent un rôle encore plus important, car le robot présente l'écran sous des orientations très variées et l'association entre blobs détectés et points physiques doit rester sans ambiguïté pour chaque pose. La figure 4.3 montre une sortie de cette détection dans la configuration de calibration extrinsèque.

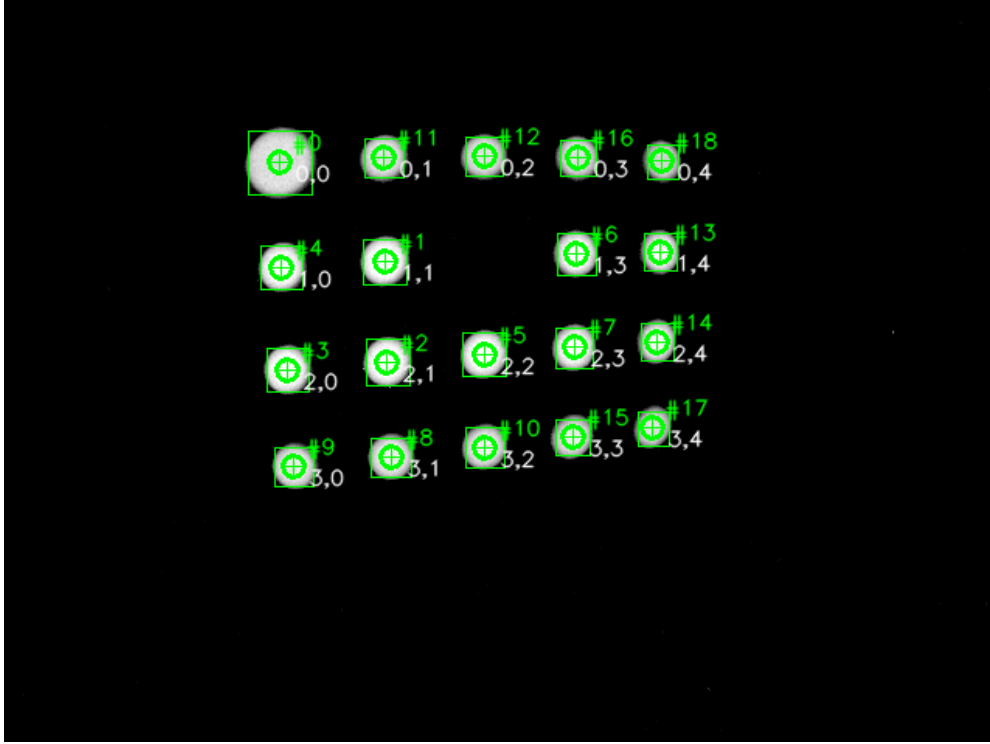


FIGURE 4.3 – Exemple de sortie de l'algorithme de détection de la mire événementielle. Les blobs détectés sont associés aux points physiques de la mire, puis annotés avec leur identifiant et leurs coordonnées dans la grille.

La principale différence avec l'utilisation en calibration intrinsèque tient aux angles d'observation. Lorsque l'écran est fortement incliné, la projection d'un disque s'allonge en ellipse et le barycentre peut se décaler légèrement par rapport au centre géométrique; un ajustement d'ellipse pourrait alors être envisagé pour affiner le centre. Le protocole retenu privilégie une solution plus simple : les acquisitions dont l'orientation détectée est jugée invalide sont rejetées avant la résolution.

4.1.5 Association 2D/3D et pose de la mire

Pour estimer la pose de la mire dans la caméra, il faut associer les centres détectés dans l'image aux points physiques connus de la mire. Les coordonnées des points dans le repère S sont coplanaires, car ils appartiennent à la surface de l'écran :

$$P_{ij}^S = \begin{bmatrix} X_{ij} \\ Y_{ij} \\ 0 \end{bmatrix}.$$

Pour une grille de cols colonnes et rows lignes, avec des espacements s_x et s_y , on peut définir :

$$X_{ij} = \left(i - \frac{\text{cols} - 1}{2}\right) s_x, \quad Y_{ij} = \left(j - \frac{\text{rows} - 1}{2}\right) s_y.$$

Ces points 3D sont mis en correspondance avec les centres image (u_{ij}, v_{ij}) . La pose ${}^C T_{S,i}$ peut alors être estimée avec une méthode de type solvePnP [12], en utilisant les paramètres intrinsèques de la caméra événementielle et, si nécessaire, les coefficients de distorsion. Une bonne calibration intrinsèque est donc un prérequis : une erreur dans les paramètres caméra se propagera directement dans la transformation extrinsèque.

4.1.6 Calibration multi-poses

La solution pose par pose permet de vérifier que le système fonctionne, mais elle ne constitue pas la méthode finale. La calibration doit plutôt chercher une transformation unique qui minimise l'erreur de reprojection sur toutes les poses robot et tous les points observés.

Pour chaque pose i et chaque point j , on connaît :

- la position du point P_j^S dans la mire ;
- la pose robot de la mire ${}^B T_{S,i}$;
- l'observation image $\hat{\mathbf{u}}_{ij}$;
- les paramètres intrinsèques de la caméra.

En cherchant ${}^C T_B$, l'erreur de reprojection globale peut s'écrire :

$$\min_{{}^C T_B} \sum_{i,j} \left\| \pi \left({}^C T_B {}^B T_{S,i} P_j^S \right) - \hat{\mathbf{u}}_{ij} \right\|^2,$$

où π représente la projection caméra. Cette formulation est plus robuste qu'une moyenne directe des estimations ${}^B T_{C,i}$, car elle ajuste la transformation finale à partir des erreurs réellement visibles dans l'image.

4.1.7 Paramètres physiques de l'écran

La mire affichée sur l'écran doit être connue en dimensions réelles. Dans l'outil Python développé, cette étape est automatisée : le programme récupère la résolution de l'écran ainsi que sa taille physique, puis en déduit directement le facteur de conversion entre pixels et millimètres. Aucune mesure manuelle de l'écran n'est donc nécessaire pour générer la mire avec les bonnes dimensions. La densité en pixels par millimètre est calculée par :

$$\text{px/mm}_x = \frac{\text{largeur en pixels}}{\text{largeur en mm}}, \quad \text{px/mm}_y = \frac{\text{hauteur en pixels}}{\text{hauteur en mm}}.$$

Le script utilise ces valeurs pour définir automatiquement le diamètre des disques, leur espacement et l'origine du repère S . Cela évite d'introduire une erreur supplémentaire liée à une mesure faite à la main, tout en gardant une correspondance directe entre les coordonnées pixels de la mire et ses coordonnées physiques en millimètres.

4.1.8 Tests de précision et points critiques

Des tests ont été réalisés avec l’outil Python afin de vérifier que la pose estimée à partir de la mire permet bien de prédire d’autres points affichés sur le même écran. Le test se déroule en deux phases. Dans un premier temps, la mire complète à 19 points est affichée et détectée, puis une pose de l’écran est calculée avec `solvePnP` [12]. Dans un second temps, le programme affiche une série de cibles carrées à quatre points, avec des positions et des tailles différentes. Les quatre coins attendus sont projetés dans l’image à partir de la pose estimée, puis comparés aux blobs réellement détectés.

Deux erreurs différentes sont alors calculées. La première est l’erreur de reprojection en pixels : chaque coin réel du carré est projeté dans l’image, puis comparé au centre du blob détecté correspondant. Le score affiché est un RMS sur ces distances image :

$$e_{\text{repr}} = \sqrt{\frac{1}{N} \sum_i \|\hat{\mathbf{u}}_i - \mathbf{u}_i\|^2}.$$

La seconde erreur est calculée en millimètres. Les quatre coins détectés sont reprojetés sur le plan estimé de l’écran, ce qui permet de reconstruire les quatre longueurs du carré. Ces longueurs mesurées sont ensuite comparées aux longueurs réelles connues :

$$e_{\text{cote}} = \frac{1}{4} \sum_{k=1}^4 |L_k^{\text{mesure}} - L_k^{\text{reel}}|.$$

Une sortie de ce test est visible sur la partie droite de la figure 3.4, déjà présentée pour la calibration intrinsèque. Sur cet essai, les quatre carrés de validation sont acceptés. L’erreur de reprojection globale est d’environ 0,13 px, tandis que l’erreur moyenne sur les longueurs reconstruites des côtés est d’environ 0,16 mm. La pose utilisée pour cette validation place l’écran à environ 634 mm de la caméra, avec une inclinaison d’environ 20°. Ces valeurs ne constituent pas encore une précision finale de calibration robot-caméra, mais elles montrent que la chaîne de détection, d’association, de projection et de reconstruction métrique est cohérente sur le plan de l’écran.

La précision finale de la calibration extrinsèque dépendra ensuite à la fois de cette détection événementielle, de la géométrie du montage mécanique et de la diversité des poses robot utilisées. L’erreur angulaire reste particulièrement importante : une erreur d’orientation de 1° représente environ 17 mm d’erreur latérale à 1 m, et environ 35 mm à 2 m. Pour une tâche d’interception de balle, cette erreur peut devenir aussi critique qu’une erreur directe sur la profondeur.

Le protocole expérimental devra donc privilégier :

- des poses nombreuses et variées, avec des orientations différentes de la mire ;
- une couverture large de l’image caméra ;
- des plans de mire non tous parallèles ;
- une correction correcte de la distorsion ;
- le rejet des poses où les blobs sont mal détectés ;
- l’analyse des résidus de reprojection après optimisation.

4.1.9 Résultats de la calibration physique caméra-robot

Ce protocole a été appliqué sur le montage réel, visible sur la figure 4.4 : le smartphone affichant la mire est fixé sur le support imprimé en 3D au bout du bras UR3e, et la caméra événementielle observe la scène depuis un trépied fixe. Le robot présente ensuite la mire dans une série de poses variées pendant que la caméra détecte les disques clignotants.

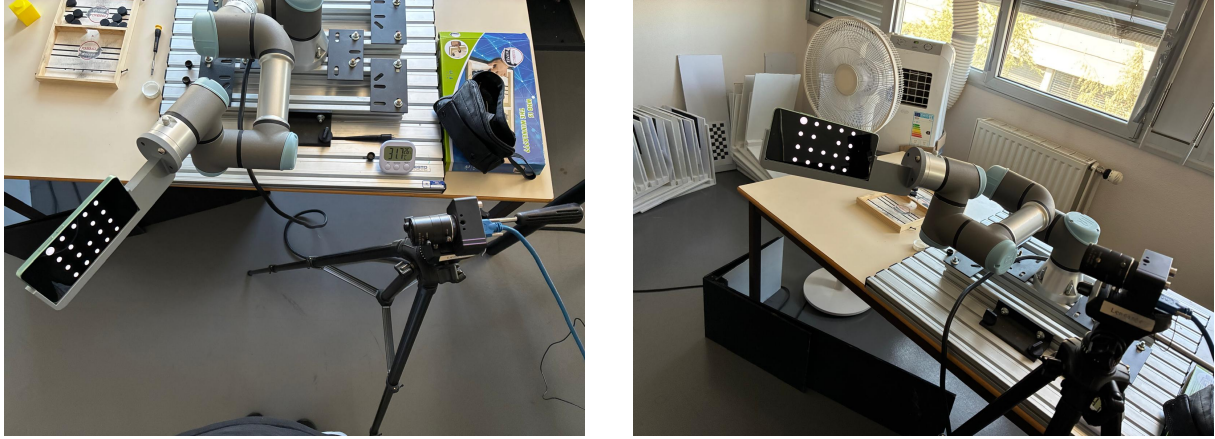


FIGURE 4.4 – Montage réel de la calibration extrinsèque. Le smartphone affichant la mire clignotante est porté par le UR3e via le support imprimé en 3D, et la caméra événementielle observe la scène depuis un trépied fixe.

Après rejet des acquisitions dont l'orientation d'écran détectée était invalide, dix-huit poses variées ont été retenues pour la résolution, conformément à la cible de quinze à vingt poses fixée par le protocole.

Cette calibration passe les critères d'acceptation numériques fixés à l'avance. Le résidu de translation entre les poses prédites et observées est de 0,94 mm en moyenne et de 2,24 mm au maximum, et le résidu de rotation de 0,25° en moyenne et de 0,44° au maximum. Une validation croisée en retirant chaque pose à tour de rôle déplace la solution d'au plus 0,36 mm et 0,05°, ce qui indique qu'aucune pose isolée ne domine le résultat. L'erreur de reprojection de bout en bout, mesurée sur 342 points de mire, est de 1,22 px en RMS avec un maximum de 3,46 px. La diversité angulaire des poses utilisées est d'environ 90°, ce qui évite une configuration dégénérée du problème. Rapportés à la sensibilité angulaire discutée plus haut, ces résidus correspondent, pour l'erreur de rotation moyenne, à un décalage latéral de l'ordre de 4 mm à 1 m, soit un ordre de grandeur en dessous des quelques centimètres qui suffisent à faire rater une interception : la précision obtenue est donc compatible avec le besoin, sous réserve des vérifications physiques indépendantes mentionnées plus bas.

La transformation obtenue est ensuite publiée dans le système robotique et restituée dans l'interface de supervision 3D, comme le montre la figure 4.5 : la caméra apparaît sous forme d'une pyramide de vue avec son repère, positionnée par rapport au modèle du robot et à l'écran porté par l'effecteur. Cette visualisation fournit une première vérification qualitative immédiate : la position et l'orientation de la caméra affichées correspondent bien au placement physique du trépied par rapport au robot. Les valeurs numériques complètes de la transformation sont données dans l'annexe E.

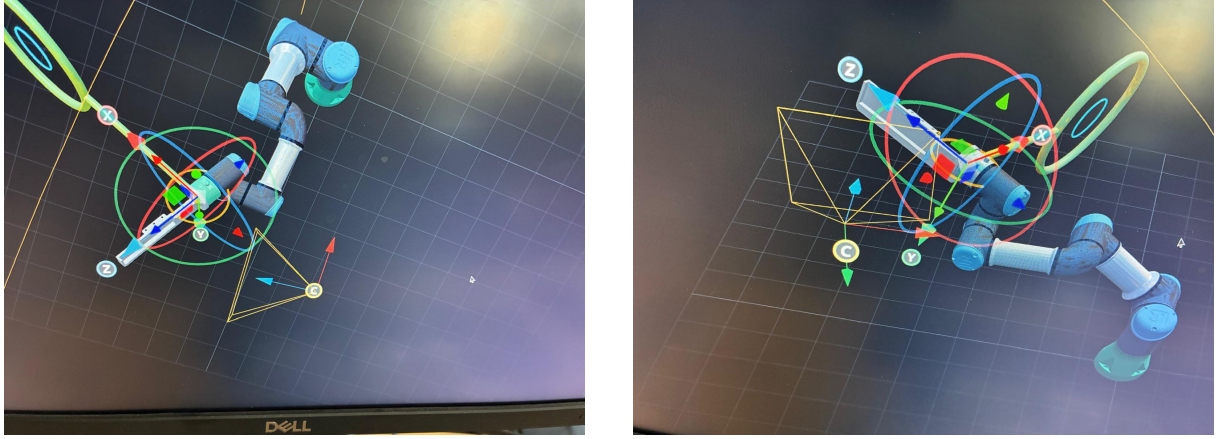


FIGURE 4.5 – Pose 3D de la caméra restituée dans l’interface de supervision après la calibration extrinsèque, sous deux angles de vue. La pyramide jaune représente le champ de vision de la caméra avec son repère ; sa position et son orientation par rapport au robot et à l’écran porté par l’effecteur sont cohérentes avec le montage physique de la figure 4.4.

Cette transformation reste propre à l’installation : déplacer la caméra ou modifier le support du téléphone l’invalidé immédiatement. Deux vérifications physiques restent à confirmer avant de considérer la calibration comme définitive : la cohérence avec des mesures indépendantes du montage (mètre ruban et modèle CAO) et la vérification quantitative de la transformation publiée en continu, notamment en comparant la position d’une même balle exprimée directement dans le repère du robot et reconstruite depuis le repère caméra.

4.1.10 Lien avec l’interception robotique

Cette calibration est l’étape qui rend la perception exploitable par le robot. Une fois ${}^B T_C$ connue, une position de balle estimée dans le repère caméra peut être transformée dans le repère de la base :

$$P^B = {}^B T_C P^C.$$

La trajectoire obtenue par la méthode Trace peut alors être exprimée dans le repère robot, filtrée, prédite, puis transmise à la commande ou à une politique d’interception. La qualité de cette transformation conditionne directement la précision finale : même une bonne estimation caméra ne suffit pas si la calibration extrinsèque déplace la balle de plusieurs centimètres dans le repère du robot.

4.2 Apprentissage d’une politique d’interception en simulation

4.2.1 Objectif de la partie robot

La perception seule ne suffit pas pour attraper la balle. Elle doit être intégrée à une commande robotique capable de déplacer l’effecteur vers une position d’interception. Cette partie a été étudiée dans Isaac Lab [15] avec un apprentissage par renforcement de type PPO [18], pour *Proximal Policy Optimization*.

L'objectif de PPO [18] est d'apprendre une politique de commande. À partir de l'état courant du robot et de la balle, la politique choisit une action qui rapproche le robot d'une situation favorable pour intercepter la balle. La simulation permet de répéter un grand nombre d'essais sans risque pour le matériel réel. La figure 4.6 montre le montage réel considéré pour relier la perception à la robotique.

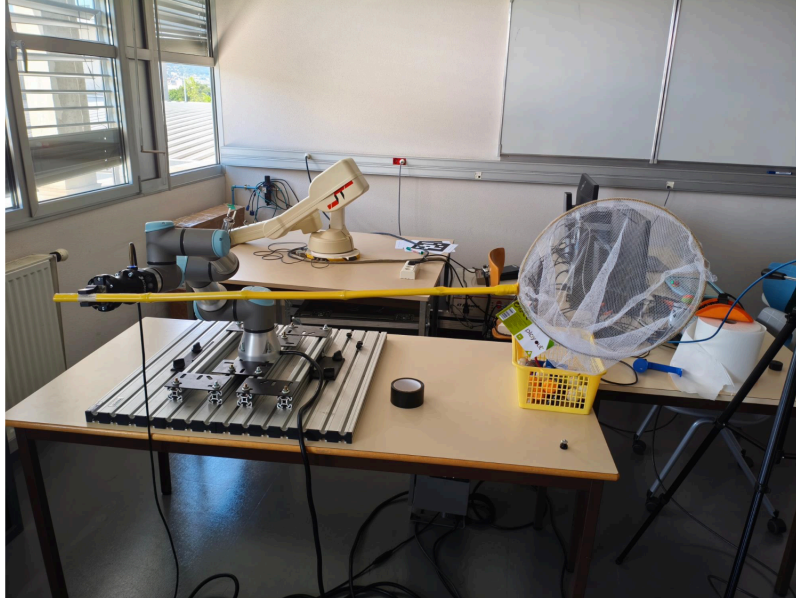


FIGURE 4.6 – Montage réel avec robot, caméra et zone de réception.

4.2.2 Environnement d'apprentissage

L'environnement simulé reproduit le robot UR3e portant le filet et une balle lancée vers lui. À chaque pas de temps, la politique reçoit une observation de 32 valeurs exprimées dans le repère de la base du robot :

- les positions et les vitesses des six articulations (12 valeurs) ;
- la position du centre du filet (3 valeurs) ;
- la position de la balle (3 valeurs) et sa vitesse (3 valeurs) ;
- la direction unitaire du filet vers la balle (3 valeurs) et la distance filet-balle (1 valeur) ;
- un indicateur du côté du plan du filet où se trouvait la balle au pas précédent, dont le changement sert à détecter un passage (1 valeur) ;
- la dernière action appliquée (6 valeurs).

Cette définition est importante pour la suite : le système réel devra construire exactement la même observation, dans le même ordre et le même repère, pour que la politique apprise reste valable.

La politique produit six actions normalisées entre -1 et 1 , interprétées comme des incréments de consigne articulaire à 60 Hz. L'incrément maximal par pas de temps correspond à la vitesse limite de l'axe, soit π rad/s pour les trois premiers axes et 2π rad/s pour les poignets, c'est-à-dire au plus environ 3° et 6° par pas respectivement. Avant d'être intégrés en une cible de position, ces incréments sont en outre limités en accélération par articulation, et la cible reste bornée sur chaque axe. Ce choix incrémental borne physiquement le mouvement demandé, quelles que soient les sorties du réseau, et les mêmes limites sont réappliquées lors du déploiement réel.

La récompense combine un terme continu qui augmente lorsque la distance entre le filet et la

balle diminue, un fort bonus lorsque la balle traverse le filet et une pénalité lorsque l'épisode se termine par un contact avec le bras ou par la chute de la balle au sol. S'y ajoute une pénalisation quadratique de l'amplitude des actions, réglée articulation par articulation : les coefficients sont environ cinq fois plus élevés sur les axes de l'épaule et du coude que sur les poignets, afin de pénaliser davantage les mouvements des axes les plus massifs, et ils augmentent progressivement pendant la première partie de l'entraînement, ce qui laisse la politique apprendre d'abord le geste d'interception avant de l'affiner en mouvements plus sobres. Deux pénalités complémentaires régularisent enfin le comportement : l'une décourage la saturation des actions au-delà de leurs bornes, l'autre les variations brusques d'action entre deux pas de temps consécutifs. Chaque épisode dure au plus 4 s. La balle apparaît entre 1,2 et 2,1 m devant le robot, à une hauteur comprise entre 0,5 et 1,2 m, et est lancée vers lui avec une vitesse principale de 3,5 à 5 m/s, une composante latérale de l'ordre de $\pm 0,7$ m/s et une composante verticale initiale pouvant atteindre 1,5 m/s, ce qui laisse un vol libre de l'ordre de 0,3 à 0,5 s pour réagir. Un bruit gaussien, d'écart-type de 1 à 5 cm selon la variante, est ajouté à la position de balle observée pour préparer la robustesse au bruit de perception, et une balle qui traverse le filet réapparaît aussitôt pour un nouveau lancer sans interrompre l'épisode. Deux variantes existent, avec le filet monté à droite ou à gauche de l'effecteur ; une politique apprise pour un côté n'est pas transférable à l'autre, et le déploiement doit donc rester cohérent avec le montage physique.

4.2.3 Déroulement et résultats de l'entraînement

L'entraînement fait progresser 12 000 environnements simulés en parallèle, par pas de 16,7 ms. La politique est mise à jour toutes les 32 étapes de collecte à partir des trajectoires accumulées, et l'état du réseau est sauvegardé régulièrement, ce qui permet de suivre la récompense moyenne par épisode au cours de l'apprentissage et de conserver le comportement retenu.

Les deux entraînements retenus, un par côté de montage du filet, suivent la même dynamique. La récompense moyenne par épisode part d'une valeur négative, dominée par les pénalités de fin d'épisode, devient positive après environ 1 500 étapes de collecte, puis atteint un plateau de l'ordre de 1 400 à 1 550 en moins de 10 000 étapes, soit environ $1,2 \times 10^8$ pas de simulation cumulés et quelques minutes de calcul. La suite de l'entraînement ne modifie plus ce plateau et sert essentiellement à en vérifier la stabilité. La variante droite a ainsi été poursuivie jusqu'à 400 000 étapes de collecte, soit $4,8 \times 10^9$ pas de simulation et 12 500 mises à jour de la politique en environ 2 h 45 ; la variante gauche a été arrêtée après environ 194 000 étapes, soit $2,3 \times 10^9$ pas en environ 1 h 20, le plateau étant atteint depuis longtemps. C'est le dernier état sauvegardé de cet entraînement gauche qui est déployé sur le robot réel.

Ces chiffres montrent que la simulation massivement parallèle permet d'apprendre l'essentiel du comportement d'interception en quelques minutes, le reste du budget de calcul servant à confirmer la stabilité du résultat. Cependant, le passage vers le système réel demande de reconstruire fidèlement, autour du robot réel, les conditions vues par la politique en simulation.

4.3 Boucle d'interception temps réel

Pour utiliser la politique sur le robot réel, une boucle de commande à 60 Hz a été mise en place. À chaque cycle, elle reçoit la dernière position de balle estimée, la transforme dans le repère de la base du robot grâce à la calibration extrinsèque, construit le vecteur d'observation de 32 valeurs décrit précédemment, exécute la politique apprise, convertit l'action en consigne articulaire avec les mêmes limites de vitesse et d'accélération qu'en simulation, puis applique une couche de sécurité indépendante avant toute transmission au robot. La consigne finale est

envoyée au contrôleur du robot sous forme d'un flux interpolé à 500 Hz, ce qui lisse le mouvement entre deux décisions de la politique.

La sécurité est traitée comme une propriété du système et non de la politique. L'émission de commande vers le robot est désactivée par défaut et doit être armée explicitement par l'opérateur. La boucle refuse de commander le robot si aucun modèle de politique n'est chargé, si la position de la balle arrive dans un repère inconnu, ou si la transformation donnant la position du filet n'est pas disponible. Un facteur global de réduction de vitesse permet une montée en régime progressive lors des essais, et un chien de garde interrompt la commande si les données nécessaires cessent d'arriver.

4.4 Premiers essais sur le robot réel

4.4.1 Validation de la chaîne de commande avec une balle virtuelle

La chaîne de commande a d'abord été validée sans caméra, en remplaçant la perception par un générateur de balles virtuelles suivant la même enveloppe de lancers que la simulation d'apprentissage. Dans cette configuration, la chaîne complète fonctionne sur le robot réel : la politique calcule les actions à partir de la balle virtuelle, le robot suit le point d'interception, puis maintient sa position une fois la balle au sol. Sous les limites de mise en service, avec le facteur de vitesse réduit de moitié, la réponse du robot reste cependant plus lente que le comportement simulé ; l'optimisation de cette réponse fait partie des travaux restants.

4.4.2 Premiers essais commandés avec la caméra réelle

Les premiers essais de la chaîne complète, avec la caméra événementielle réelle, la méthode Trace et la commande du robot activée, ont mis en évidence un défaut d'architecture plutôt que de perception : le robot ne produisait que des tressaillements et l'état de commande oscillait dans l'interface. Le diagnostic a montré que deux instances de la boucle de décision et deux sources de position de balle fonctionnaient simultanément, la pile de test à balle virtuelle étant restée active à côté de la chaîne réelle. Les messages invalides intercalés par la source virtuelle inactive déclenchaient des arrêts contrôlés répétés, d'où les tressaillements.

Des protections en ont été tirées : la boucle détecte désormais la présence de plusieurs producteurs sur les données de balle et bloque l'émission de commande dans ce cas, l'interface signale les oscillations de l'état de commande, et la procédure de lancement a été réorganisée autour d'une pile logicielle unique. Cet incident illustre un enseignement important du stage : sur un système robotique réel, la fiabilité de l'architecture système, notamment l'unicité des sources de données et la discipline des procédures de lancement, est aussi critique que la qualité des algorithmes.

4.5 Défis pour l'intégration perception-robot

Le tableau 4.1 récapitule les principaux défis identifiés pour passer d'une perception fonctionnelle à une interception fiable, et sert de grille de lecture pour l'état d'avancement du projet.

TABLE 4.1 – Principaux défis pour passer de la perception à l’interception robotique.

Défi	Impact attendu
Latence perception-commande	Une estimation correcte mais trop tardive peut déplacer le point d’interception. Il faut mesurer le temps entre acquisition, traitement, publication ROS 2 [11] et action robot.
Synchronisation temporelle	La trajectoire doit être associée à des timestamps fiables. Sans synchronisation, le robot peut suivre une balle déjà passée.
Calibration caméra-robot	La position 3D estimée dans le repère caméra doit être transformée dans le repère du robot avec une extrinsèque précise.
Espace atteignable	Toutes les trajectoires de balle ne sont pas interceptables. Il faut vérifier la zone de travail, les limites articulaires et la vitesse maximale du robot.
Bruit de mesure	Une erreur de quelques centimètres en profondeur peut suffire à rater la balle. La méthode Trace réduit ce risque, mais un filtrage temporel reste nécessaire.
Sim-to-real	Une politique apprise en simulation peut échouer si la dynamique réelle, les délais ou les observations diffèrent trop du simulateur.
Sécurité	Les essais réels doivent limiter les vitesses, gérer les arrêts d’urgence et éviter les trajectoires dangereuses du robot.

La plupart de ces défis ont maintenant une réponse au moins partielle : la calibration extrinsèque est réalisée et validée numériquement, la chaîne de commande fonctionne sur le robot réel avec une balle virtuelle, et les mécanismes de sécurité ont été éprouvés lors des premiers essais. Les points encore ouverts concernent la mesure des latences réelles de la chaîne, la validation quantitative de la perception réelle et l’optimisation de la vitesse de réponse du robot.

Chapitre 5

Conclusion

Bilan technique

L’objectif central du stage, obtenir une position 3D exploitable d’une balle rapide à partir d’une caméra événementielle, a été atteint et dépassé sur plusieurs points. Deux méthodes d’estimation ont été développées et comparées. La méthode par ajustement de cercle a fourni une première solution fonctionnelle, mais son analyse a mis en évidence une limite structurelle : la profondeur dépend entièrement d’un rayon apparent très sensible à la vitesse de la balle. La méthode par trace événementielle, construite pour répondre à cette limite, exploite la forme réellement produite par la balle dans le flux d’événements et s’est montrée plus adaptée aux lancers rapides. La validation sur des séquences simulées avec vérité terrain a permis de séparer les erreurs algorithmiques des difficultés expérimentales.

Au-delà de la perception, le stage a couvert l’ensemble de la chaîne d’interception. Deux outils de calibration adaptés aux caméras événementielles ont été développés, pour les paramètres intrinsèques puis pour la transformation caméra-robot, avec des erreurs résiduelles de l’ordre du millimètre et de quelques dixièmes de degré sur le montage réel. Une boucle de commande temps réel relie la perception à une politique d’interception apprise par renforcement, avec une architecture de sécurité indépendante de la politique. La chaîne complète a été validée sur le robot réel avec une balle virtuelle, et les premiers essais avec la caméra réelle ont été réalisés. L’interception complète d’une balle réelle n’est pas encore démontrée : elle dépend des validations restantes sur les latences, la qualité de la perception réelle et la vitesse de réponse du robot, qui constituent la suite naturelle du projet.

Bilan personnel

Ce stage m’a permis de travailler sur un système mécatronique complet, du capteur jusqu’au mouvement du robot, dans un contexte de recherche appliquée. J’ai appris à raisonner sur des compromis concrets : fenêtre temporelle contre réactivité, robustesse contre latence, généralité contre performance temps réel. Les incidents rencontrés lors des essais réels ont été particulièrement formateurs : ils m’ont montré que la mise en service d’un robot repose autant sur la rigueur des procédures, l’analyse des journaux et la discipline de l’architecture système que sur les algorithmes eux-mêmes. Enfin, ce stage m’a donné une vision claire du travail de recherche expérimentale en robotique, où chaque affirmation doit être appuyée par une mesure, et où la sécurité du matériel et des personnes structure l’ensemble de la démarche.

Annexe A

Expressions des régressions de trajectoire par moindres carrés

Cette annexe détaille les expressions utilisées pour ajuster les modèles de trajectoire présentés dans la partie perception. Les coordonnées horizontales sont modélisées par une droite et la coordonnée verticale par une parabole ; les coefficients sont calculés directement à partir de sommes accumulées au fil des mesures, sans résolution matricielle générale.

Pour une coordonnée linéaire $q \in \{X, Y\}$, avec n observations (t_i, q_i) :

$$D = n \sum t_i^2 - \left(\sum t_i \right)^2,$$
$$a = \frac{n \sum t_i q_i - (\sum t_i) (\sum q_i)}{D}, \quad b = \frac{\sum q_i - a \sum t_i}{n}.$$

Pour la coordonnée quadratique Z , le modèle est $q(t) = at^2 + bt + c$. Les grandeurs intermédiaires utilisées sont :

$$S_{11} = \sum t_i^2 - \frac{(\sum t_i)^2}{n}, \quad S_{12} = \sum t_i^3 - \frac{(\sum t_i) (\sum t_i^2)}{n},$$
$$S_{22} = \sum t_i^4 - \frac{(\sum t_i^2)^2}{n}, \quad S_{q1} = \sum t_i q_i - \frac{(\sum q_i) (\sum t_i)}{n},$$
$$S_{q2} = \sum t_i^2 q_i - \frac{(\sum q_i) (\sum t_i^2)}{n}, \quad D_q = S_{22} S_{11} - S_{12}^2.$$

Les coefficients quadratiques sont alors :

$$b = \frac{S_{q1} S_{22} - S_{q2} S_{12}}{D_q}, \quad a = \frac{S_{q2} S_{11} - S_{q1} S_{12}}{D_q}, \quad c = \bar{q} - b\bar{t} - a\bar{t}^2.$$

La version pondérée conserve les mêmes modèles mais remplace chaque somme simple par la somme pondérée correspondante, avec les poids de récence et de robustesse définis dans le corps du rapport.

Annexe B

Pipeline détaillé de la détection par circle fitting

Cette annexe détaille les étapes internes de la méthode par circle fitting, dont la vue simplifiée est donnée par la figure 3.6. La figure B.1 décompose les trois phases du traitement : la réduction du flux d'événements, la validation du cluster avec l'ajustement du cercle, puis le calcul de la pose 3D et de la trajectoire.

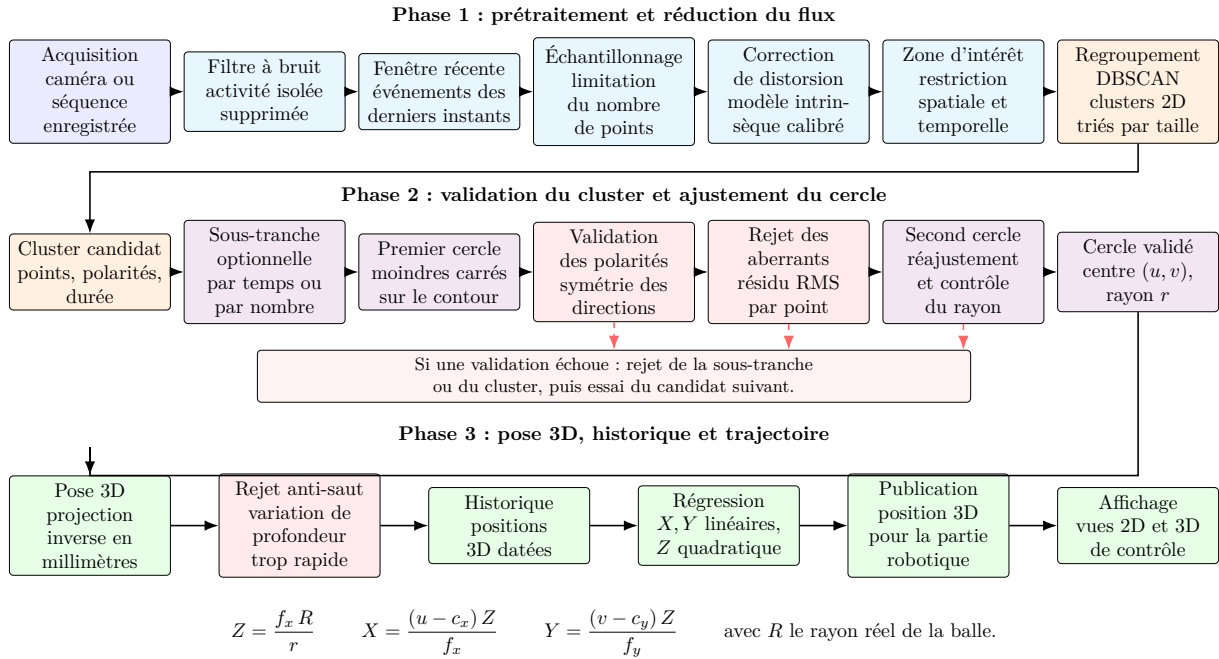


FIGURE B.1 – Pipeline détaillé de la détection 3D par circle fitting, en trois phases : réduction du flux d'événements, validation du cluster et ajustement du cercle, puis pose 3D et trajectoire. Les formules du bas donnent la conversion du cercle image (u, v, r) en position 3D dans le repère caméra.

Annexe C

Détails de la reconstruction du ruban de trace

Cette annexe détaille les critères et les expressions utilisés par la méthode par trace événementielle pour reconstruire le ruban, dont les étapes principales sont résumées dans la partie perception.

Repère local de la trace

Le centre c du nuage est estimé de manière robuste avec les médianes des coordonnées, puis les points trop éloignés sont retirés à partir d'un quantile radial. Une PCA globale fournit ensuite la direction principale d de la trace. Pour les trajectoires courbes, cette direction est corrigée par des PCA temporelles calculées sur plusieurs sous-parties de la trace ; l'orientation finale combine la direction globale, la moyenne pondérée des directions locales et le déplacement temporel entre le début et la fin de la trace. Le repère local est défini par :

$$s = (p - c) \cdot d, \quad h = (p - c) \cdot n,$$

où p est un événement image et $n = (-d_y, d_x)$ est la normale à la trace. Dans les schémas de cette annexe, l'axe h est dessiné verticalement comme une hauteur ; il s'agit en réalité de la coordonnée transversale à la trace : l'étendue des événements selon h correspond donc à la largeur locale du ruban, celle mesurée dans chaque bin.

Bins spatiaux

La trace est découpée en bins spatiaux le long de s . Chaque bin j produit une estimation locale unique, composée de trois valeurs :

$$(s_j, h_{\text{bas},j}), \quad (s_j, h_{\text{milieu},j}), \quad (s_j, h_{\text{haut},j}), \quad \text{avec} \quad h_{\text{milieu},j} = \frac{h_{\text{haut},j} + h_{\text{bas},j}}{2}.$$

La position s_j du bin est une valeur représentative des coordonnées s de ses événements, par exemple la médiane, et la largeur locale vaut $w_j = h_{\text{haut},j} - h_{\text{bas},j}$.

Détection des bords soutenus

Prendre directement $h_{\text{bas}} = \min h_i$ et $h_{\text{haut}} = \max h_i$ serait trop sensible au bruit : un seul événement isolé deviendrait un faux bord et fausserait la largeur, donc la profondeur. Un bord n'est donc accepté que s'il est soutenu par assez d'événements voisins. Le support requis n_{support} est une fraction du nombre d'événements du bin, bornée entre un minimum et un maximum fixés.

Les valeurs de h sont triées, puis balayées depuis chaque extrémité. Pour le bord bas, le balayage part du plus petit h et retient le premier candidat h_k dont la fenêtre $[h_k, h_k + r_s]$ contient au moins n_{support} événements. Le bord haut est obtenu de manière symétrique, en partant des grandes valeurs de h . La figure C.1 illustre cette logique : les points isolés aux extrémités ne sont pas retenus comme bords s'ils ne possèdent pas assez de voisins proches.

Recherche des bords par support local sur l'axe h

Les bords ne sont pas les minimums/maximums : ils doivent avoir assez de voisins proches.

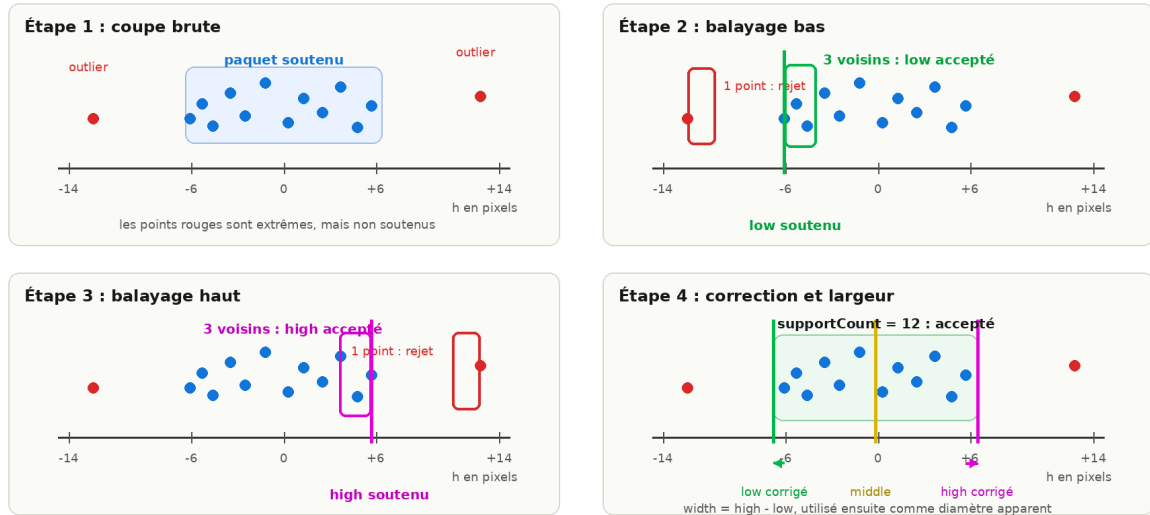


FIGURE C.1 – Recherche locale des bords par support sur l'axe h . Les événements isolés sont ignorés ; seuls les extrêmes possédant assez de voisins proches deviennent h_{bas} et h_{haut} . L'écart entre ces deux bords, dessiné ici comme une hauteur, est la largeur locale du bin.

Comme les événements correspondent aux centres des pixels, le bord apparent peut se trouver légèrement au-delà du dernier événement soutenu : les deux bords sont donc écartés d'une petite correction proportionnelle à la largeur locale et bornée à 1,5 px. La coupe est enfin validée : elle est rejetée si elle contient trop peu d'événements, si $h_{\text{haut}} \leq h_{\text{bas}}$, si la largeur brute est trop faible, ou si le nombre d'événements compris entre les deux bords finaux reste inférieur au support minimal.

Filtrage des estimations locales

Le filtrage se fait en deux niveaux. Le premier vérifie la cohérence globale du ruban : un modèle polynomial simple, linéaire ou quadratique selon le nombre d'échantillons, est ajusté sur les lignes médianes, et un bin est rejeté si sa ligne médiane est trop éloignée de la tendance générale ou si sa largeur locale est trop différente de la largeur médiane du ruban.

Le second niveau vérifie la cohérence locale. Pour chaque bin j , une petite régression sur les bins voisins, par exemple $j - 4$ à $j + 4$ en ignorant le bin courant pour éviter qu'il ne se valide lui-même, prédit les valeurs attendues $\hat{h}_{\text{bas},j}$, $\hat{h}_{\text{milieu},j}$, $\hat{h}_{\text{haut},j}$ et $\hat{w}_j$. Le bin est supprimé s'il casse la continuité locale :

$$|w_j - \hat{w}_j| > \tau_w, \quad |h_{\text{bas},j} - \hat{h}_{\text{bas},j}| > \tau_{\text{bas}}, \quad |h_{\text{milieu},j} - \hat{h}_{\text{milieu},j}| > \tau_{\text{milieu}}, \quad |h_{\text{haut},j} - \hat{h}_{\text{haut},j}| > \tau_{\text{haut}},$$

où les seuils τ dépendent de la largeur médiane du ruban et de la dispersion locale, estimée par une MAD. Cette étape évite deux erreurs opposées : garder un bin totalement incohérent qui ferait exploser la largeur, donc la profondeur ; ou supprimer un bin simplement parce qu'il s'écarte de la moyenne globale alors qu'il reste cohérent avec son voisinage. La figure C.2 résume ce principe.

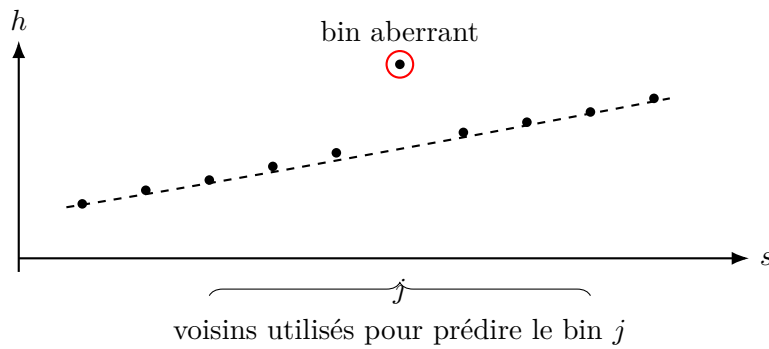


FIGURE C.2 – Principe du filtrage local. Le bin courant est comparé à une prédiction construite avec ses voisins ; s'il crée une rupture trop forte, il est supprimé.

Courbes du ruban

Après filtrage, il reste trois listes d'échantillons $\{(s_j, h_{\text{bas},j})\}$, $\{(s_j, h_{\text{milieu},j})\}$ et $\{(s_j, h_{\text{haut},j})\}$. Les trois courbes sont ajustées indépendamment, ce qui laisse le ruban souple : le bord haut et le bord bas ne sont pas forcés d'avoir exactement la même forme, ce qui est utile si la trace est asymétrique ou si une polarité domine localement.

L'ajustement est local : autour d'une position s_0 , les échantillons proches pèsent davantage que les échantillons éloignés. Pour un ajustement local quadratique centré sur s_0 :

$$h(s) = a(s - s_0)^2 + b(s - s_0) + c, \quad h(s_0) = c.$$

Les coefficients sont recalculés pour plusieurs positions s_0 le long du ruban, et la fenêtre locale peut être élargie dans les zones pauvres en échantillons afin de conserver une estimation stable. Chaque point estimé est ensuite reconverti dans le repère image avec :

$$p(s_0) = c_{\text{trace}} + s_0 d + h(s_0) n,$$

où c_{trace} est le centre du ruban, d sa direction principale et n sa normale. La figure C.3 illustre cet ajustement.

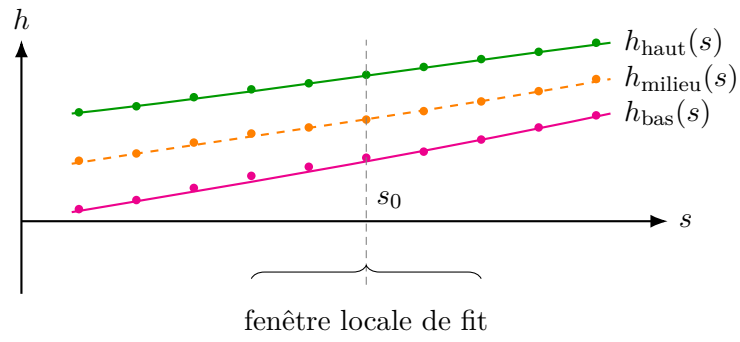


FIGURE C.3 – Les trois courbes du ruban sont ajustées à partir des échantillons valides. L’ajustement est local : autour de s_0 , les points proches pèsent davantage que les points éloignés.

Annexe D

Paramètres des séquences simulées

Les séquences de validation présentées dans la partie perception ont été générées avec Isaac Sim [14], puis converties en flux événementiel avec `v2e` [16, 17], avec les paramètres suivants :

- résolution vidéo : 640×480 ;
- fréquence d'entrée : 500 fps ;
- rayon de balle : 0.02 m ;
- calibration synthétique : $f_x = f_y = 520$ px, $c_x = 320$ px, $c_y = 240$ px ;
- résolution temporelle des événements : $200 \mu s$;
- seuils de génération ON/OFF : 0.15.

Annexe E

Résultats numériques de la calibration caméra-robot

Cette annexe regroupe les valeurs numériques de la calibration extrinsèque caméra-robot décrite dans la partie intégration robotique. Les longueurs sont en mètres et les quaternions sont donnés dans l'ordre (x, y, z, w) .

La pose de la caméra dans le repère de la base du robot est :

$$t_{B \rightarrow C} = \begin{bmatrix} 0,4294 \\ -0,5946 \\ 0,4345 \end{bmatrix} \text{ m}, \quad q_{B \rightarrow C} = \begin{bmatrix} 0,4319 \\ 0,5815 \\ -0,5498 \\ -0,4160 \end{bmatrix},$$

soit une orientation d'environ $(-92,8^\circ, -0,5^\circ, 106,3^\circ)$ en angles roulis-tangage-lacet.

La transformation entre l'effecteur du robot et le centre de la zone active de l'écran, issue de la géométrie du support et des mesures de bords, est :

$$t_{E \rightarrow S} = \begin{bmatrix} -0,0053 \\ -0,0006 \\ 0,1588 \end{bmatrix} \text{ m}, \quad q_{E \rightarrow S} = \begin{bmatrix} -0,0002 \\ 0,7118 \\ -0,0007 \\ 0,7024 \end{bmatrix}.$$

Pour cette seconde transformation, la représentation en angles d'Euler est à éviter : le tangage proche de 90° place la conversion au voisinage du blocage de cardan, et seule la forme quaternion est numériquement fiable.

Les critères de validation associés à cette calibration sont les suivants :

TABLE E.1 – Métriques de validation de la calibration extrinsèque caméra-robot.

Métrique	Valeur
Poses retenues	18
Résidu de translation (moyen / max)	0,94 mm / 2,24 mm
Résidu de rotation (moyen / max)	0,25° / 0,44°
Sensibilité au retrait d'une pose (pire cas)	0,36 mm / 0,05°
Erreur de reprojection (RMS / max, 342 points)	1,22 px / 3,46 px
Diversité angulaire des poses	$\approx 90^\circ$

L'estimation de la position de la zone active de l'écran dans le téléphone, utilisée pour construire la transformation effecteur-écran, repose sur les mesures de bords suivantes :

TABLE E.2 – Estimation des bords entre le téléphone et la zone active de l'écran.

Élément	En pixels	En millimètres estimés
Bord gauche	≈ 23 px	$\approx 2,92$ mm
Bord droit	≈ 21 à 22 px	$\approx 2,67$ à $2,80$ mm
Bord haut	≈ 23 px	$\approx 2,93$ mm
Bord bas	≈ 26 px	$\approx 3,31$ mm

Annexe F

Récapitulatif des réalisations du stage

Cette annexe récapitule, par thème, les principales réalisations produites pendant le stage. Chacune est décrite et analysée dans le corps du rapport ; la liste sert de vue d'ensemble de la contribution personnelle.

Perception événementielle

- Algorithme de détection 3D par circle fitting : filtrage et réduction du flux d'événements, clustering, ajustement de cercle avec validations de polarité et de symétrie, calcul de la pose 3D et régression de trajectoire.
- Algorithme de détection 3D par trace événementielle : reconstruction du ruban laissé par la balle, estimation locale des largeurs, conversion 3D et régression pondérée, conçu pour répondre aux limites du circle fitting sur les balles rapides.
- Interface de visualisation temps réel développée avec raylib [13] : affichage des événements, des détections et des trajectoires en vues 2D et 3D, avec les outils de diagnostic utilisés pour l'analyse des erreurs.

Calibration

- Logiciel de calibration intrinsèque par mire événementielle affichée sur écran : génération de la mire clignotante, détection des disques, calibration robuste et test de validation par cibles carrées.
- Logiciel de calibration extrinsèque caméra-robot : détection de la mire portée par le robot, résolution multi-poses de la transformation caméra-base et métriques de validation associées.
- Conception et impression 3D du support fixant le smartphone porte-mire à l'effecteur du robot.

Simulation et apprentissage

- Génération de séquences événementielles simulées avec vérité terrain : balle lancée dans Isaac Sim [14], conversion du rendu en flux d'événements, utilisée pour valider quantitativement les deux méthodes de perception.
- Entraînement par renforcement PPO [18] d'une politique d'interception dans Isaac Lab [15], avec les deux variantes de montage du filet.

Intégration robotique

- Boucle d'interception temps réel à 60 Hz reliant la perception à la politique apprise, avec limites articulaires, couche de sécurité indépendante et chien de garde.
- Générateur de balles virtuelles permettant de valider la chaîne de commande complète sans caméra.
- Interface de supervision et de contrôle : visualisation 3D en direct du robot et de la balle, suivi du mouvement commandé, restitution de la calibration extrinsèque, armement de la commande, réglage du facteur de vitesse et lancement de balles virtuelles.
- Conception et impression 3D du support de maintien du filet d'interception sur l'effecteur.

Bibliographie

- [1] Antonio Claudio De Sousa Dos Santos Filho, *Computer Vision, Reinforcement Learning and Skill Transfer in Robotic : Application to a Dynamic Task*, rapport de stage, document fourni.
- [2] CAP 20-25, *Le Centre International de Recherche “Innovative Transportation and Production Systems” - CIR ITPS*. <https://cap2025.fr/fr/challenge2>
- [3] CAP 20-25, *Useful information of CIR ITPS*. <https://cap2025.fr/.../useful-information-of-cir-itps>
- [4] Université Clermont Auvergne, *Site institutionnel de l’Université Clermont Auvergne*. <https://www.uca.fr/>
- [5] Institut Pascal, *Présentation de l’Institut Pascal*, site officiel. <http://www.institutpascal.uca.fr/index.php/fr/l-institut-pascal/presentation>
- [6] Institut Pascal, *Axe Image, Systèmes de Perception, Robotique (ISPR)*, site officiel. <http://www.institutpascal.uca.fr/index.php/fr/presentation-ispr>
- [7] Institut Pascal, *Équipe MACCS – Modélisation, Autonomie et Contrôle dans les Systèmes Complexes*, site officiel. <http://www.institutpascal.uca.fr/index.php/fr/maccs>
- [8] Institut Pascal, *Équipe ComSee – Computers that See*, site officiel. <http://www.institutpascal.uca.fr/index.php/fr/comsee>
- [9] iniVation, *DVXplorer – Hardware documentation*, documentation officielle. <https://docs.inivation.com/hardware/current-products/dvxpplorer.html>
- [10] iniVation, *Documentation officielle iniVation*. <https://docs.inivation.com/>
- [11] Open Robotics, *ROS 2 Documentation*, documentation officielle. <https://docs.ros.org/>
- [12] OpenCV, *Camera Calibration and 3D Reconstruction – calib3d module*, documentation officielle. https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html
- [13] raylib, *raylib – A simple and easy-to-use library to enjoy videogames programming*, site officiel. <https://www.raylib.com/>
- [14] NVIDIA, *Isaac Sim – Robotics Simulation and Synthetic Data Generation*, site officiel. <https://developer.nvidia.com/isaac/sim>
- [15] NVIDIA, *Isaac Lab – Robot Learning Framework*, site officiel. <https://developer.nvidia.com/isaac/lab>
- [16] SensorsINI, *v2e : From video frames to DVS events*, dépôt GitHub. <https://github.com/SensorsINI/v2e>
- [17] Hu, Y., Liu, S.-C. et Delbruck, T., *v2e : From Video Frames to Realistic DVS Events*, CVPR Workshops, 2021. https://openaccess.thecvf.com/content/CVPR2021W/EventVision/html/Hu_v2e_From_Video_Frames_to_Realistic_DVS_Events_CVPRW_2021_paper.html
- [18] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. et Klimov, O., *Proximal Policy Optimization Algorithms*, arXiv, 2017. <https://arxiv.org/abs/1707.06347>